
Documentação de Projeto

para o sistema

Stream Sentry

Versão 1.5

Projeto de sistema elaborado pelo aluno Rafael Parreira Chequer e apresentado ao curso de Engenharia de Software da PUC Minas como parte do Trabalho de Conclusão de Curso (TCC) sob orientação dos professores Cleiton Silva Tavares, Danilo de Quadros Maia Filho, Leonardo Vilela Cardoso e Raphael Ramos Dias Costa.

14 de dezembro de 2025

Tabela de Conteúdo

Tabela de Conteúdo	2
Histórico de Revisões	2
1. Introdução	1
2. Modelos de Usuário e Requisitos	1
2.1 Descrição de Atores	1
2.2 Modelos de Usuários	2
2.3 Modelo de Casos de Uso e Histórias de Usuários	4
2.4 Diagrama de Sequência do Sistema e Contrato de Operações	6
3. Diagramas	10
3.1 Diagrama de Classes	10
3.2 Diagramas de Sequência	11
3.3 Diagramas de Comunicação	13
3.4 Arquitetura Lógica	15
3.5 Diagrama de Atividade	16
3.6 Diagrama de Componentes e Diagrama de Implantação	17
4. Projeto de Interface com Usuário	20
4.1 Interfaces Comuns a Todos os Atores	20
5. Modelos de Projeto	27
5.1 Modelo de Dados	28
5.2 Glossário	29
6. Plano de Testes	31
6.1 Plano de Teste de Aceitação	31
6.2 Plano de Teste de Integração	37
7. Cronograma e Processo de Implementação	38
7.1 Cronograma	38
7.2 Metodologia de Desenvolvimento	41
7.3 Ferramentas e Pipeline de CI/CD	41

Histórico de Revisões

Nome	Data	Razões para Mudança	Versão
Rafael Parreira Chequer	17/09/2025	Criação inicial do documento para entrega da A3, contendo diagrama de casos de uso, modelos de usuários e projeto de tela.	1.0

Nome	Data	Razões para Mudança	Versão
Rafael Parreira Chequer	13/10/2025	Criação do diagrama de sequência de sistema, diagrama de classes, diagrama de sequência e diagrama de comunicação e adição das demais interfaces.	1.1
Rafael Parreira Chequer	02/11/2025	Adição da arquitetura lógica, diagrama de atividades, componentes e implantação. Inclusão do modelo de dados e do glossário.	1.2
Rafael Parreira Chequer	18/11/2025	Adição do cronograma.	1.3
Rafael Parreira Chequer	30/11/2025	Adição dos planos de teste, metodologia de desenvolvimento, ferramentas e pipeline de CI/CD.	1.4
Rafael Parreira Chequer	14/12/2025		1.5

1. Introdução

Este documento apresenta a análise de usuário, contexto, interface e interação do sistema Stream Sentry, uma ferramenta web *open-source* (licença MIT) desenvolvida em React para automação de testes *end-to-end* em aplicações de videoconferência baseadas em *WebRTC* ou *APIs* de videoconferência (ex.: Zoom SDK, Jitsi). A referência principal para a descrição do problema, domínio e requisitos é o Documento de Visão (anexo a este documento), que detalha o escopo, as funcionalidades, os usuários-alvo (desenvolvedores React, engenheiros de QA (*Quality Assurance*), equipes ágeis e pesquisadores acadêmicos) e as restrições do sistema.

2. Modelos de Usuário e Requisitos

Esta seção apresenta os modelos de usuário e os requisitos funcionais do sistema Stream Sentry, com base nas necessidades identificadas no Documento de Visão. São descritos os atores envolvidos, personas representativas, casos de uso, histórias de usuário, diagramas de interação e contratos de operações, garantindo alinhamento entre as expectativas dos usuários e as funcionalidades do sistema. O objetivo é fornecer uma base clara e estruturada para o desenvolvimento, priorizando usabilidade, automação e extensibilidade em cenários de teste de videoconferência.

2.1 Descrição de Atores

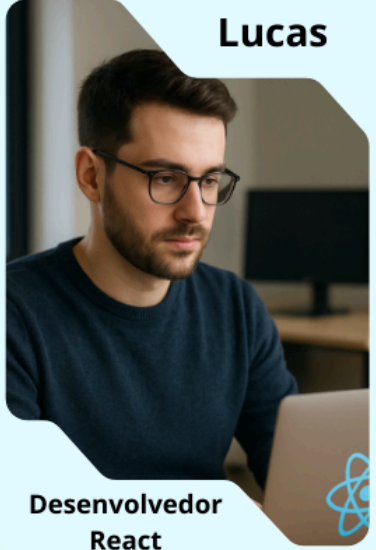
Os atores que interagem com o sistema Stream Sentry são definidos com base nos usuários-alvo descritos no Documento de Visão. Abaixo está a descrição de cada ator:

- **Desenvolvedor React:** Profissional que desenvolve aplicações de videoconferência em React, utilizando *WebRTC* ou *APIs* como Zoom SDK ou Jitsi. Este ator utiliza o Stream Sentry para automatizar testes *end-to-end*, reduzindo o tempo e o esforço em validações manuais.
- **Engenheiro de QA:** Responsável por garantir a qualidade de aplicações de videoconferência, utilizando o sistema para executar testes automatizados e analisar relatórios detalhados com métricas como latência, falhas e cobertura de testes.
- **Equipe Ágil:** Grupo de profissionais (desenvolvedores, testadores e gerentes) que utilizam o sistema em fluxos de trabalho ágeis para integrar testes automatizados e relatórios em seus processos.

2.2 Modelos de Usuários

Para representar os usuários do sistema, foram desenvolvidas personas que refletem as características, necessidades e objetivos dos atores identificados. As personas são baseadas nas necessidades levantadas no Documento de Visão e ajudam a guiar o design do sistema.

Persona 1:



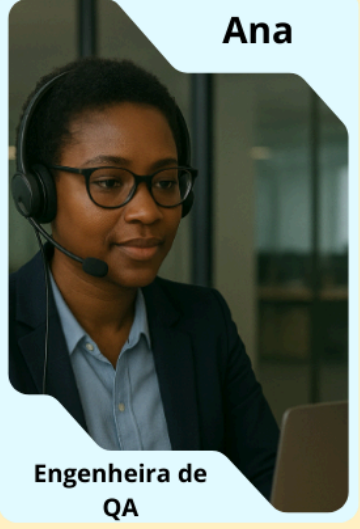
Lucas

**Desenvolvedor
React**

- **Idade:** 28 anos
- **Formação:** Bacharel em Ciência da Computação
- **Contexto:** Trabalha em startup desenvolvendo aplicações WebRTC
- **Objetivos:** Automatizar testes end-to-end para estabilidade e redução de bugs
- **Necessidades:** Interface simples, relatórios JSON/CSV, suporte a WebRTC
- **Frustrações:** Testes manuais demorados, ferramentas pagas caras
- **Cenário de Uso:** Configura teste com 5 usuários virtuais no StreamSentry e exporta relatório JSON

Figura 1: Lucas Desenvolvedor React

Persona 2:



Ana

**Engenheira de
QA**

- **Idade:** 32 anos
- **Formação:** Engenharia de Software
- **Contexto:** Valida qualidade de áudio/vídeo em APIs como Jitsi
- **Objetivos:** Executar testes automatizados com métricas de latência e falhas
- **Necessidades:** Relatórios detalhados, interface intuitiva, suporte a headless browsers
- **Frustrações:** Ferramentas como Selenium não otimizadas para WebRTC
- **Cenário de Uso:** Configura teste de estabilidade com 10 usuários e exporta relatório CSV

Figura 2: Ana Engenheira de QA

Persona 3:



Felipe

Tech Lead

- **Idade:** 35 anos
- **Formação:** Engenharia de Software com MBA em Gestão Ágil
- **Contexto:** Tech Lead de startup de videoconferência, coordenando CI/CD
- **Objetivos:** Automatizar testes WebRTC, reduzir tempo de feedback de qualidade
- **Necessidades:** Integração com CircleCI, suporte Kubernetes, notificações Teams
- **Frustrações:** Testes falhando com codecs, configuração em containers complexa
- **Cenário de Uso:** Integra StreamSentry no pipeline e valida chamadas.

Figura 3: Lucas Tech Lead

2.3 Modelo de Casos de Uso e Histórias de Usuários

O diagrama de casos de uso do sistema Stream Sentry foi desenvolvido para representar as interações principais dos atores com o sistema, conforme as funcionalidades descritas no Documento de Visão. O diagrama está disponível na pasta Artefatos do repositório com o nome UseCaseDiagram.puml e é apresentado na Figura 1.

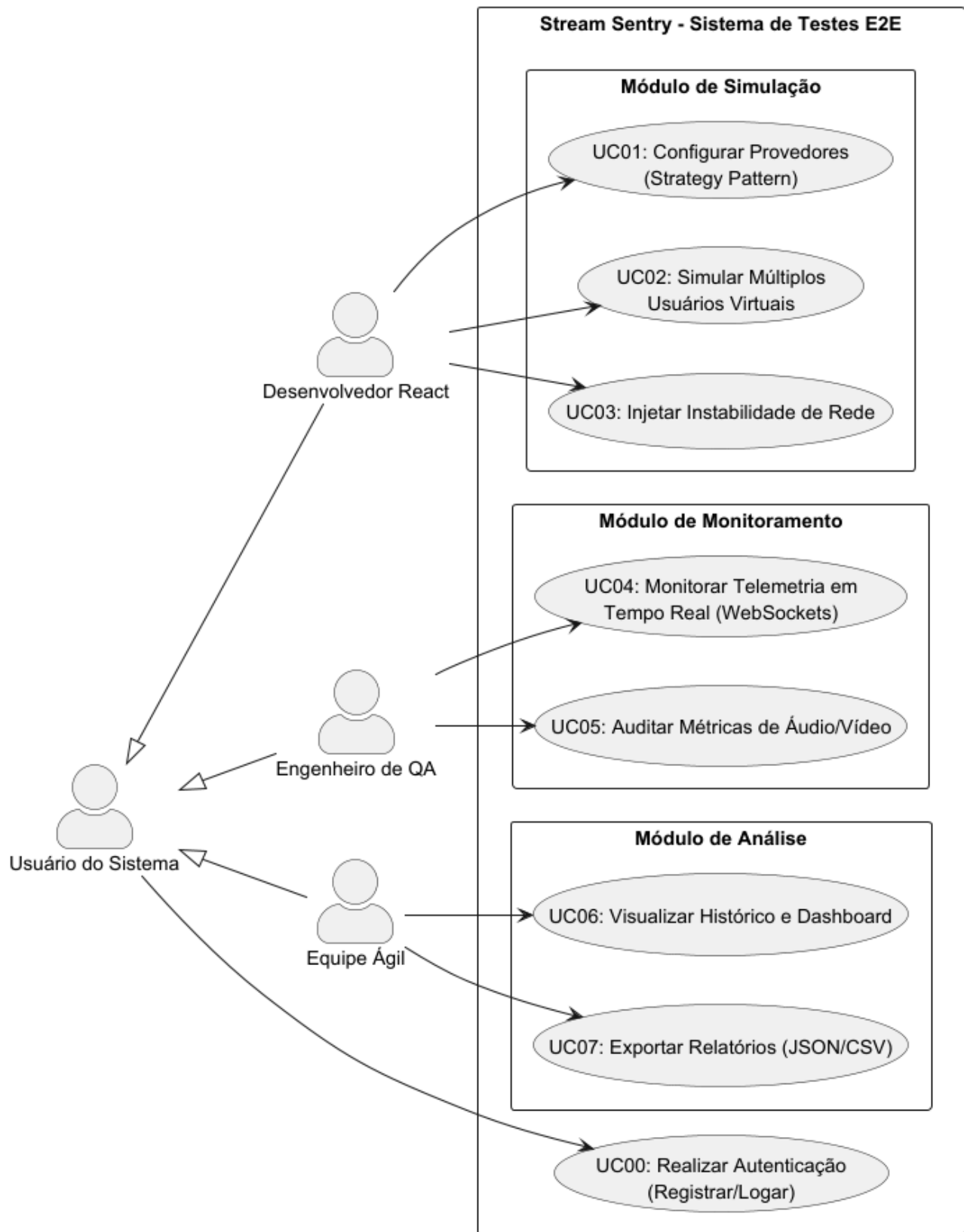


Figura 4: Diagrama de Casos de Uso do Stream Sentry

O diagrama ilustra os atores (Desenvolvedor React, Engenheiro de QA e Equipe Ágil) e suas interações com os casos de uso que sustentam o ecossistema do Stream Sentry. Cada interação reflete uma funcionalidade crítica voltada para a automação de simulações, o monitoramento de fluxos de mídia e a análise de dados para tomada de decisão.

Abaixo está a lista de histórias de usuário mapeadas, utilizando identificadores únicos para referência:

- **US00:** Como usuário do sistema, eu quero realizar o registro e login, para garantir que minhas configurações de teste e relatórios sejam armazenados de forma segura e privada.
- **US01:** Como Desenvolvedor React, eu quero configurar provedores de vídeo via *Strategy Pattern*, para garantir a compatibilidade com WebRTC, Zoom e Jitsi sem acoplamento rígido.
- **US02:** Como Desenvolvedor React, eu quero simular múltiplos usuários virtuais e injetar instabilidades de rede, para verificar a resiliência da aplicação em cenários críticos de carga.
- **US03:** Como Engenheiro de QA, eu quero monitorar a telemetria em tempo real via WebSockets, para identificar falhas de latência ou interrupções instantaneamente durante a execução.
- **US04:** Como Engenheiro de QA, eu quero auditar métricas de áudio e vídeo, para validar a qualidade da experiência (QoE) antes da entrega do software.
- **US05:** Como membro de uma Equipe Ágil, eu quero visualizar o histórico de testes no dashboard, para acompanhar a estabilidade do produto ao longo do ciclo de desenvolvimento.
- **US06:** Como membro de uma Equipe Ágil, eu quero exportar relatórios detalhados em JSON ou CSV, para compartilhar dados técnicos e integrar os resultados a ferramentas externas de análise.

2.4 Diagrama de Sequência do Sistema e Contrato de Operações

Esta seção descreve a dinâmica operacional do Stream Sentry, detalhando as interações entre os atores e o ecossistema do sistema. A modelagem foi atualizada para contemplar a arquitetura de microsserviços lógicos, a comunicação bidirecional via WebSockets para telemetria de alta fidelidade e os mecanismos de injeção de instabilidade de rede.

Nesta etapa, o foco reside na apresentação dos Diagramas de Sequência do Sistema (DSS). Um DSS é um artefato da UML (Unified Modeling Language) que ilustra, para um cenário específico de um caso de uso, os eventos gerados por atores externos, sua ordem e os eventos internos ao sistema. Diferente de um diagrama de sequência detalhado, o DSS trata o sistema como uma "caixa-preta", omitindo a complexidade de classes ou componentes internos e focando estritamente no contrato de interface entre o usuário e a solução tecnológica.

O estágio inicial compreende a definição das premissas de carga e estabilidade. Através da interface de configuração, o Desenvolvedor React estipula o volume de usuários virtuais e o provedor de vídeo alvo. Neste momento, o sistema aplica o padrão Strategy para validar a compatibilidade da

API (WebRTC, Zoom ou Jitsi) e persiste o cenário no banco de dados, garantindo que o motor de execução receba uma configuração íntegra e pronta para o disparo.

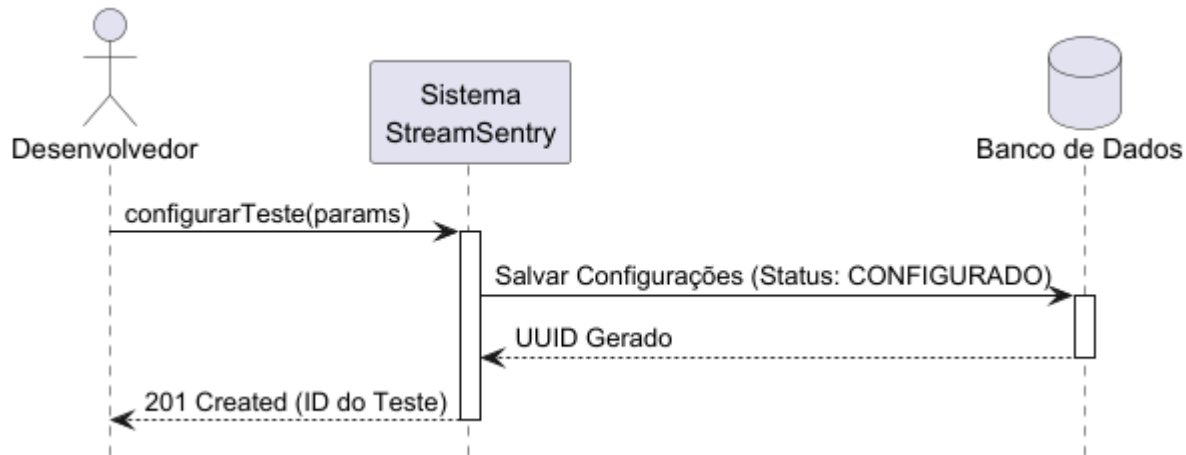


Figura 5: Diagrama de Sequência para Configuração de Cenário de Teste

A Tabela 1 formaliza a operação `configurarTeste`, ponto de entrada para a definição de cenários de carga. É indispensável que o Desenvolvedor React esteja autenticado e que o provedor de vídeo alvo seja validado pela `ProviderFactory`. Como pós-condição, o sistema assegura a integridade dos parâmetros — como volume de usuários e regras de instabilidade — persistindo-os no banco de dados sob um identificador único (UUID) e deixando o motor de execução em estado de prontidão imediata.

Contrato	Configurar Teste de Carga e Estabilidade
Operação	<code>configurarTeste(quantidadeUsuarios: int, api: string, duracao: int, instabilidade: JSON)</code>
Referências cruzadas	US01, US05, US06
Pré-condições	- Usuário está autenticado. - Provedor de vídeo validado pela Factory.
Pós-condições	- Parâmetros salvos no PostgreSQL - UUID gerado. - Status "CONFIGURADO"

Tabela 1: Contrato da Operação de Configuração

A fase de execução é caracterizada pela orquestração de Workers Puppeteer que simulam o comportamento humano em ambientes headless. Durante este ciclo, o sistema estabelece um canal de dados persistente que transmite métricas de performance (latência, bitrate e falhas) instantaneamente para o Dashboard. O diferencial deste estágio é a capacidade de intervenção do Engenheiro de QA, que pode acionar gatilhos de instabilidade para avaliar a resiliência da aplicação sob estresse de rede.

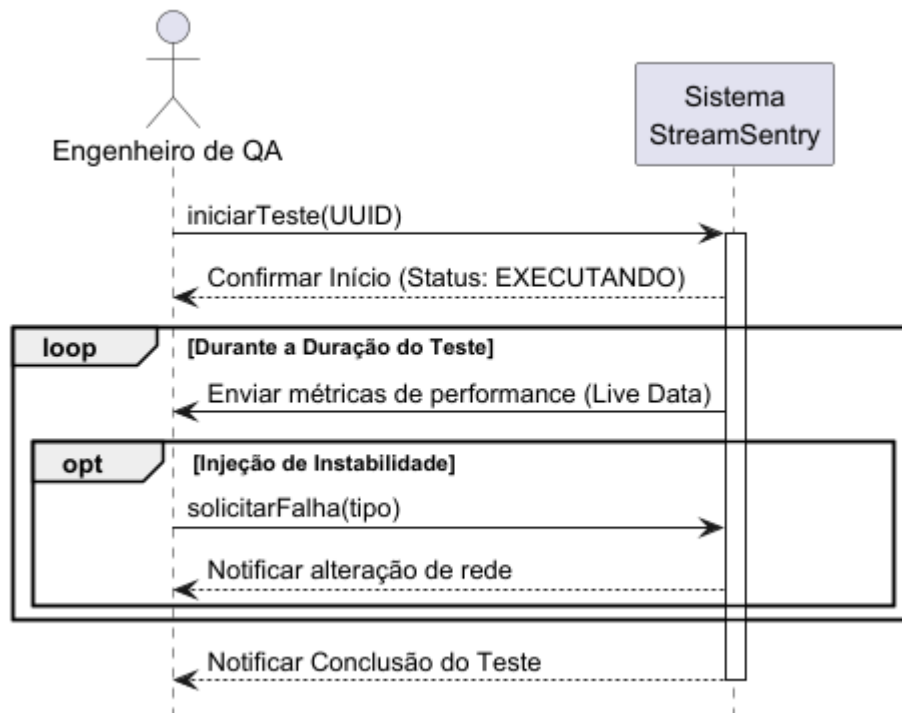


Figura 6: Diagrama de Sequência para Execução e Monitoramento

O contrato detalhado na Tabela 2 define o rigor da função executarTeste, responsável por disparar os Workers Puppeteer. As pré-condições exigem uma configuração válida e conexão ativa com a API externa. As pós-condições garantem que a simulação foi realizada conforme o planejado, com métricas de performance (latência e falhas) coletadas continuamente via WebSockets. Ao final, o sistema realiza o teardown das instâncias e atualiza o status do teste para "CONCLUÍDO".

Contrato	Executar Simulação e Telemetria em Tempo Real
Operação	executarTeste(idTeste: UUID)
Referências cruzadas	US02, US05
Pré-condições	- Teste configurado com UUID válido. - Canal de WebSocket pronto.
Pós-condições	- Simulação executada. - Telemetria persistida. - Status "CONCLUÍDO".

Tabela 2: Contrato da Operação de Análise e Exportação

O fechamento do fluxo ocorre com a transição dos dados da telemetria em tempo real para o processamento de dados históricos. O sistema agrega as séries temporais coletadas, gerando *insights* sobre a Qualidade de Experiência (QoE). Focado na *Developer Experience* (DX), esta etapa permite a exportação de relatórios estruturados (JSON/CSV), viabilizando a auditoria de resultados pela Equipe Ágil e a integração dos dados com ferramentas externas de análise ou esteiras de CI/CD.

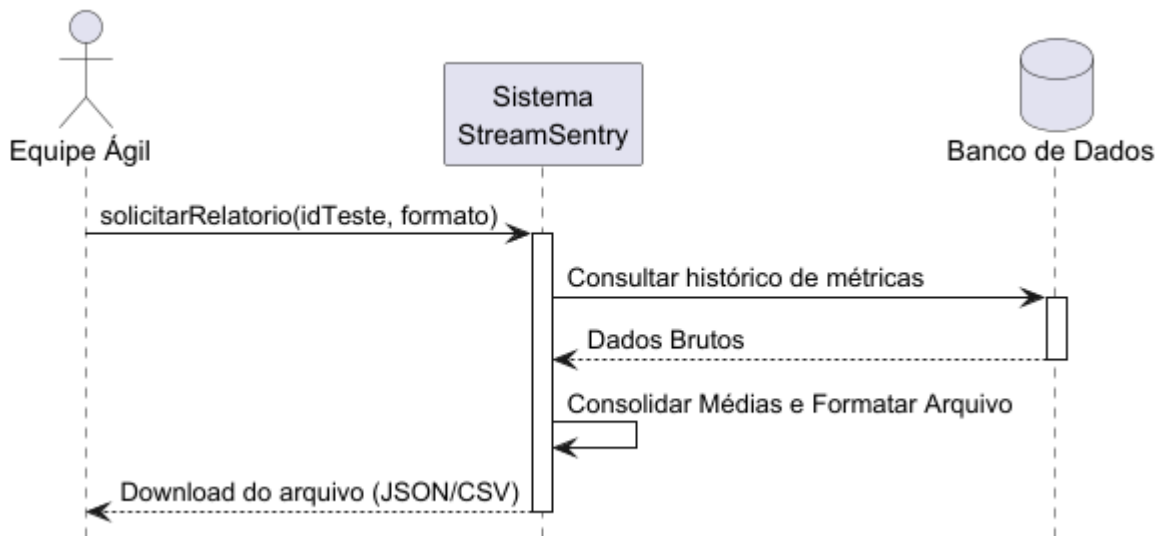


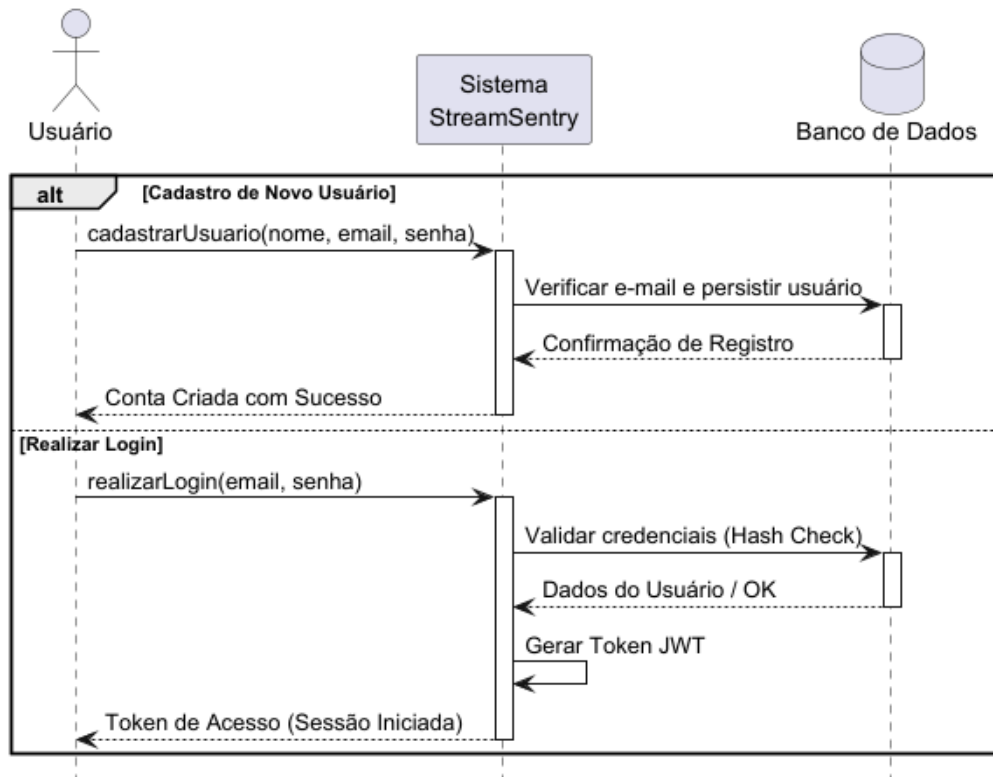
Figura 7: Diagrama de Sequência para Exportação de Dados Analíticos

A Tabela 3 estabelece as regras para a operação visualizarRelatorio, que governa a transição da telemetria em tempo real para o processamento histórico. O contrato exige que o teste tenha sido finalizado, permitindo que o sistema agregue as séries temporais para calcular a Qualidade de Experiência (QoE). O resultado é a entrega de um relatório técnico em formatos estruturados (JSON/CSV), garantindo a interoperabilidade com ferramentas externas de análise e esteiras de CI/CD.

Contrato	Gerar e Exportar Relatório Analítico de Performance
Operação	visualizarRelatorio(idTeste: UUID, formato: string)
Referências cruzadas	US03, US06
Pré-condições	- Teste concluído. - Formato de saída (JSON ou CSV) suportado.
Pós-condições	- Dados agregados em relatório técnicos. - Arquivo disponibilizado para exportação.

Tabela 3: Contrato da Operação de Visualizar Relatório

Este fluxo consolida a porta de entrada do sistema, cobrindo desde a criação da conta até a obtenção do token de acesso necessário para as demais operações.



A Tabela 0 formaliza as regras de acesso ao Stream Sentry, servindo como pré-requisito técnico para a execução de qualquer teste ou consulta de relatório.

Contrato	Realizar Cadastro e Autenticação
Operação	cadastrarUsuario(nome, email, senha) / realizarLogin(email, senha)
Referências cruzadas	US00
Pré-condições	- Para Cadastro: E-mail único no sistema. - Para Login: Credenciais válidas previamente registradas.
Pós-condições	- Usuário persistido com senha criptografada. - Token de acesso (JWT) gerado e devolvido ao frontend. - Estado do sistema alterado para "Autenticado", liberando acesso ao Dashboard.

Tabela 0: Contrato da Operação de Autenticação

3. Diagramas

A seção de Diagramas usa artefatos UML para apresentar uma visão integrada da estrutura, comportamento e arquitetura do Stream Sentry, cobrindo desde a modelagem estática até os fluxos dinâmicos e a implantação. Esses modelos sustentam a implementação, promovem a separação de responsabilidades e facilitam a compreensão, manutenção e escalabilidade do sistema em todos os seus níveis.

3.1 Diagrama de Classes

O Diagrama de Classes do Stream Sentry apresenta uma arquitetura organizada para suportar a simulação de múltiplos usuários e telemetria em tempo real, estruturada nos pacotes lógicos de Modelos (Domínio), Core (Lógica de Execução) e Integração & Estratégia. A classe TestEngine atua como o orquestrador central e núcleo do sistema, gerenciando uma lista de WorkerNode. Estes WorkerNodes são unidades isoladas responsáveis por realizar o spawn de instâncias individuais, coletar estatísticas de RTC e executar a injeção de falhas (instabilidades) de rede de forma granular.

O padrão Strategy é formalizado pela interface IConferenceProvider, que define o contrato único para as implementações de WebRTC, Zoom e Jitsi. Essa interface permite que a ProviderFactory instancie dinamicamente os drivers solicitados pelos WorkerNodes sem gerar acoplamento rígido. Para garantir a atualização instantânea do dashboard, a classe SocketManager atua como um componente transversal, sendo utilizada pelo TestEngine para dados ao vivo e pelos WorkerNodes para o envio de telemetria bruta (raw). Enquanto as entidades de domínio Usuario, Teste e Relatorio mantêm a lógica de estado, progresso e persistência no banco de dados, a arquitetura garante a separação de responsabilidades ao permitir que o Relatorio consolide as métricas finais para exportação em formatos estruturados.

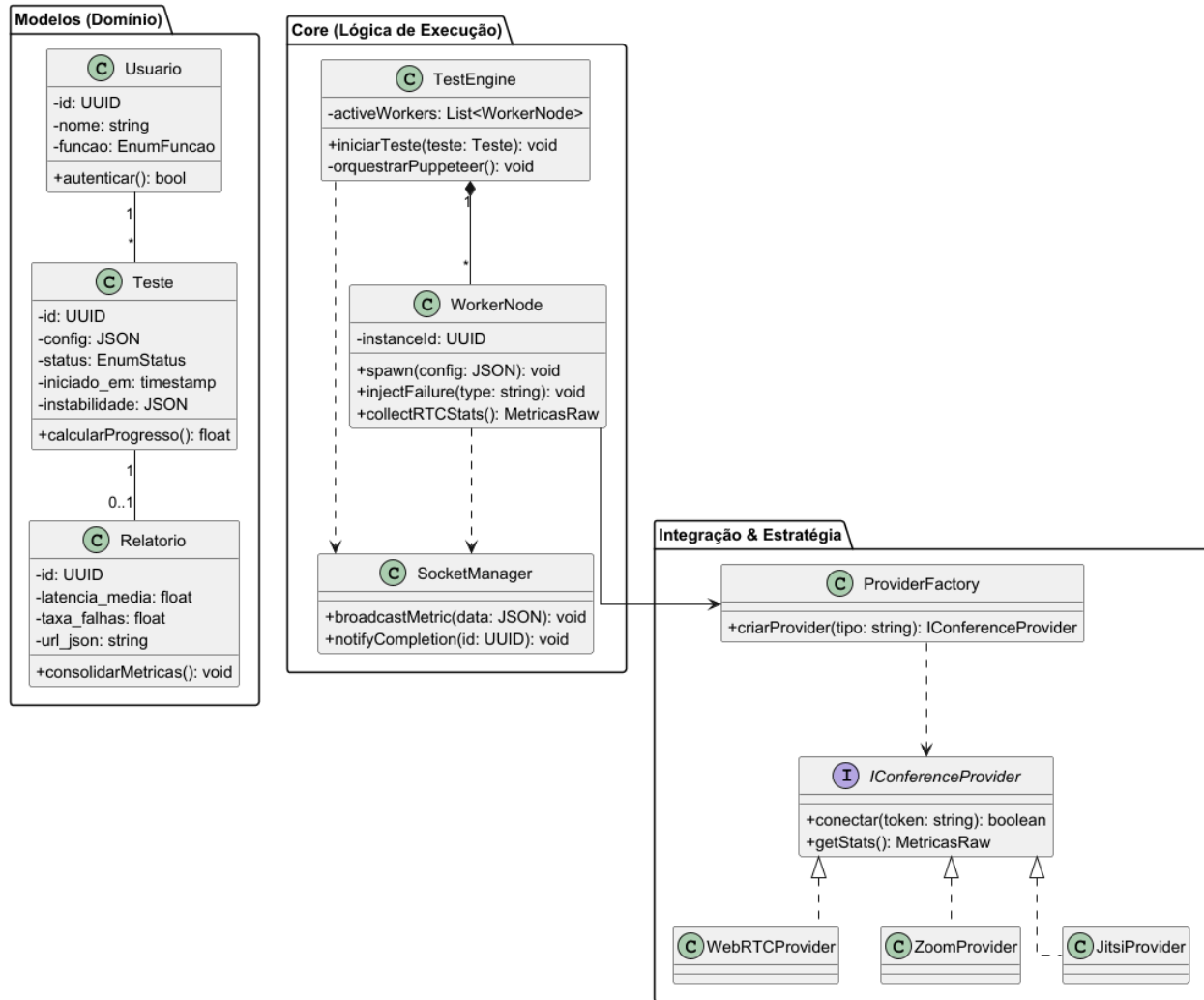


Figura 8: Diagrama de Classes do Stream Sentry

3.2 Diagramas de Sequência

Os diagramas apresentados nesta seção detalham a colaboração entre os componentes internos do Stream Sentry, revelando a lógica de orquestração e o processamento assíncrono que sustenta as funcionalidades do sistema.

O primeiro diagrama detalha o fluxo de Configuração e Validação, onde a integridade dos dados é assegurada por meio do desacoplamento de provedores. O processo inicia-se na interface React, que encaminha os parâmetros ao Core (Node.js). Ao receber a requisição, o sistema utiliza a `ProviderFactory` para instanciar o driver correspondente através da interface `IConferenceProvider`. Esta validação lógica garante que o teste só avance se o driver da API solicitada (WebRTC, Zoom ou Jitsi) estiver disponível e operacional. Somente após essa confirmação, o registro é persistido no PostgreSQL com o status "CONFIGURADO" e um identificador único (UUID), garantindo a rastreabilidade necessária para as etapas seguintes.

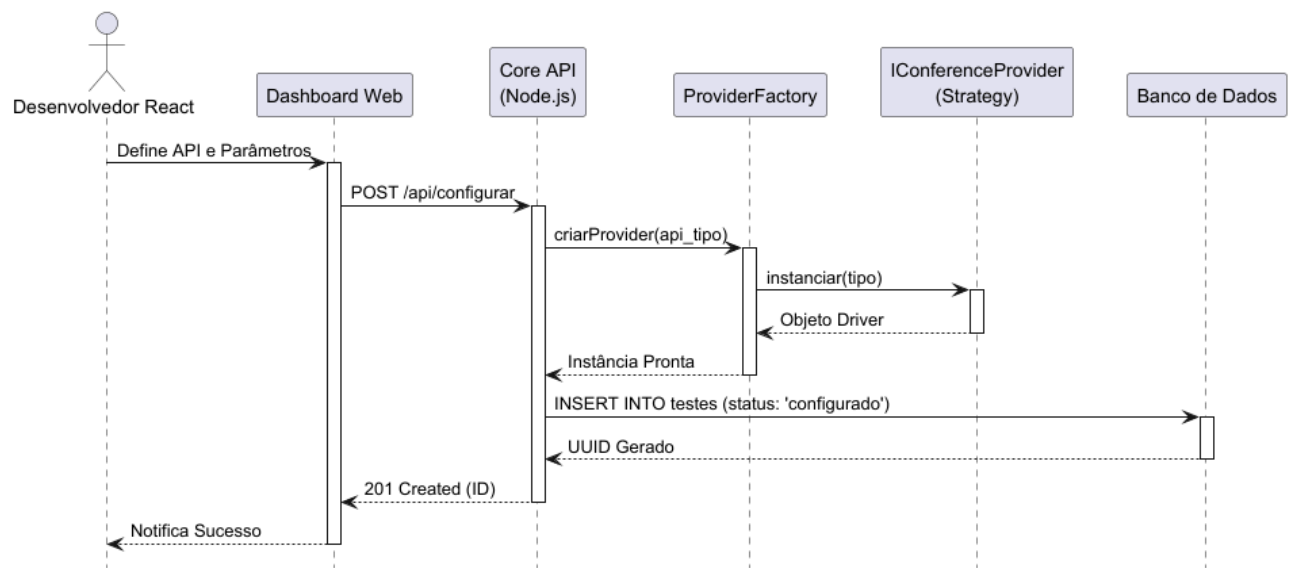


Figura 9: Diagrama de Sequência de Configuração e Validação

O segundo fluxo ilustra a Execução em Tempo Real, evidenciando a arquitetura orientada a eventos necessária para a simulação de carga. Ao receber o evento de início via WebSockets, o TestEngine assume a coordenação do ciclo de vida da simulação, realizando o spawn de instâncias isoladas de WorkerNodes via Puppeteer. Cada worker estabelece sua própria sessão de vídeo e entra em um loop de monitoramento contínuo. O diferencial reside na coleta de estatísticas de baixo nível, que são encaminhadas ao SocketManager para difusão imediata (broadcast) ao Dashboard. Simultaneamente, o Core realiza a persistência das séries temporais no banco de dados, permitindo que a visualização ao vivo ocorra sem interrupções ou bloqueios de processamento.

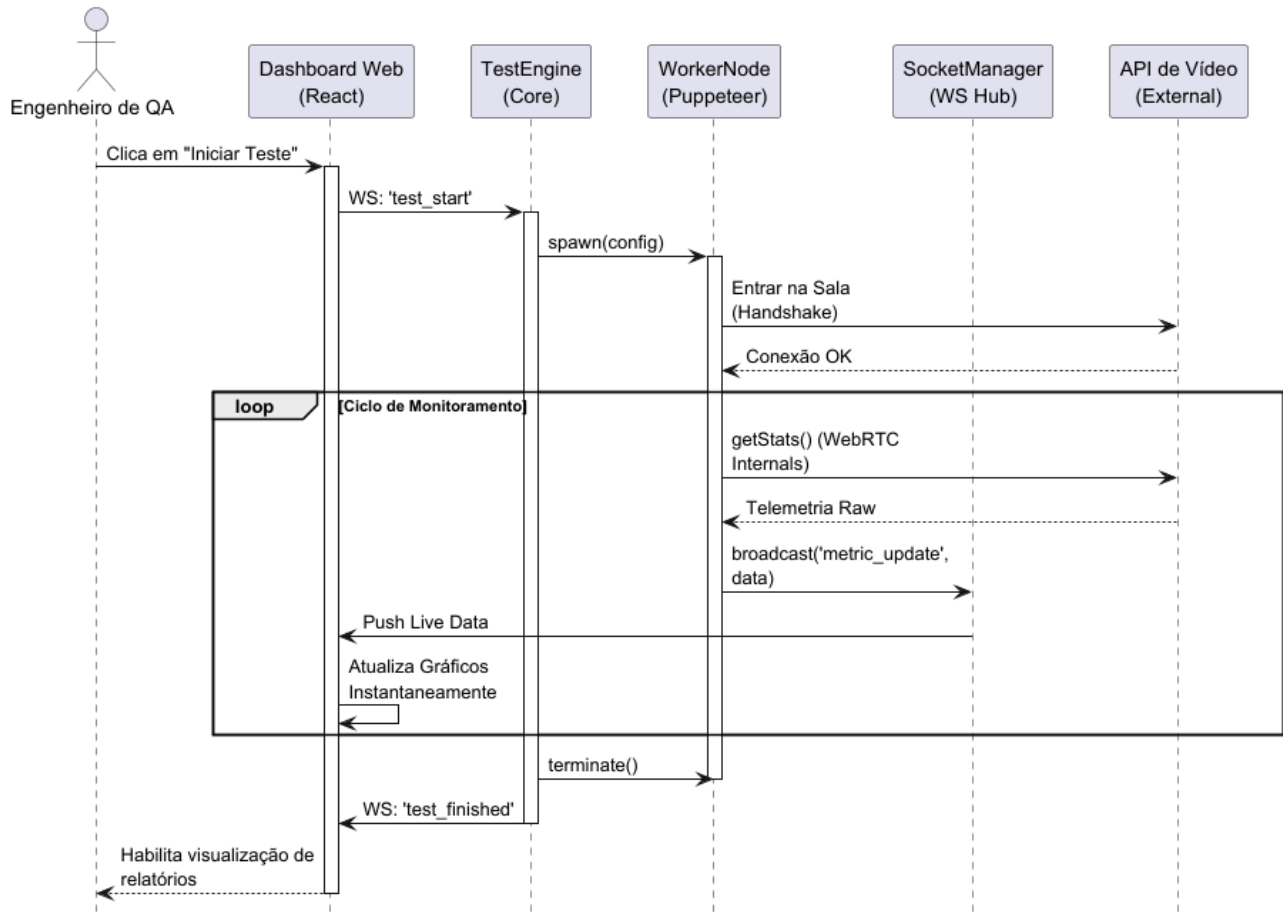
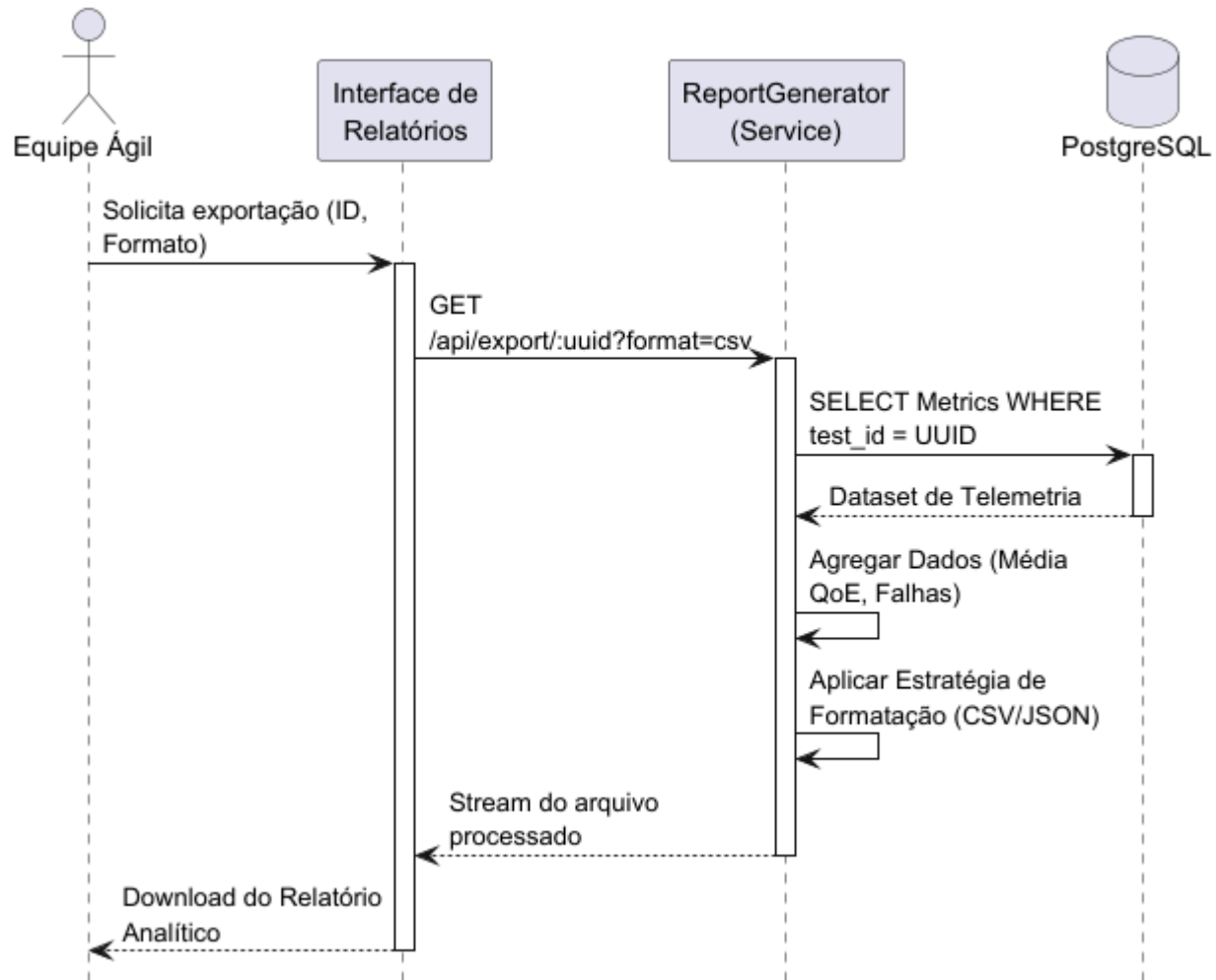


Figura 10: Diagrama de Sequência de Execução em Tempo Real

Por fim, o fluxo de Geração de Relatórios demonstra a aplicação do princípio de Separação de Responsabilidades (SoC) no tratamento de dados históricos. Este processo é acionado sob demanda, isolando o processamento analítico do motor de execução principal. O ReportGenerator executa consultas de agregação complexas no banco de dados para consolidar a telemetria armazenada, calculando métricas de Qualidade de Experiência (QoE), como latência média e taxas de falha. Utilizando estratégias de formatação específicas, o componente converte o dataset processado em arquivos JSON ou CSV e os transmite via stream para o usuário. Essa abordagem garante que a extração de grandes volumes de dados não impacte a performance de outros testes que possam estar em execução simultânea.

**Figura 11: Diagrama de Sequência de Geração de Relatórios**

3.3 Diagramas de Comunicação

O diagrama de comunicação de Configuração e Validação ilustra o estágio inicial de preparação do ambiente de teste, onde a flexibilidade arquitetural é o requisito principal. O fluxo demonstra a colaboração entre o Core API e os padrões de projeto Factory e Strategy, evidenciando como o sistema desacopla a lógica de negócio das implementações específicas de videoconferência (WebRTC, Zoom ou Jitsi). As mensagens de 3 a 6 revelam uma validação em tempo real: o sistema não apenas salva parâmetros, mas interage com a interface IConferenceProvider para garantir que o driver solicitado esteja operacional antes de persistir o teste no banco de dados. Essa estrutura assegura que apenas cenários tecnicamente viáveis avancem para a fase de execução, reduzindo o desperdício de recursos computacionais.

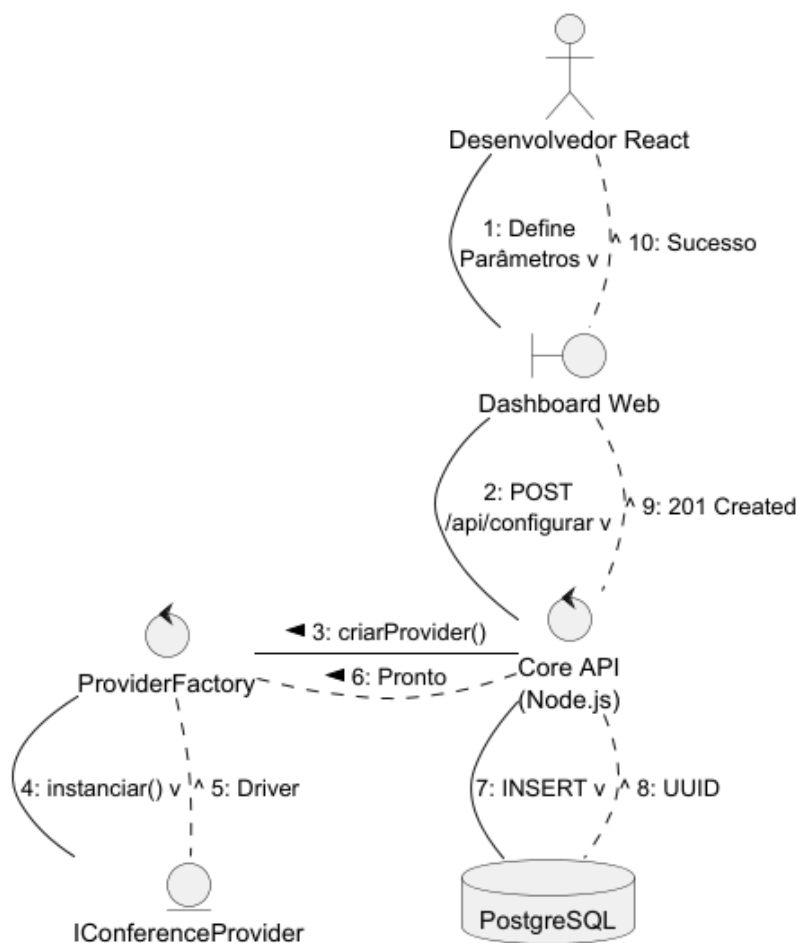


Figura 12: Diagrama de Comunicação de Configuração e Validação

O diagrama de comunicação de Execução e Telemetria foca na dinâmica do "caminho quente" (hot path), onde a baixa latência é prioritária. Ele revela uma estrutura estelar onde o TestEngine (Control) atua como o ponto central de decisão, coordenando a interação entre a interface do usuário e os WorkerNodes (Entity). A numeração das mensagens destaca o ciclo de vida assíncrono: após o spawn dos processos Puppeteer, estabelece-se um fluxo contínuo de dados onde as mensagens de telemetria bruta não sobrecarregam o motor principal. Em vez disso, o SocketManager distribui esses eventos em tempo real, permitindo que o Dashboard reflita instantaneamente o estado da rede de vídeo, enquanto a persistência em banco de dados ocorre em paralelo, garantindo que o monitoramento live seja fluido e não-bloqueante.

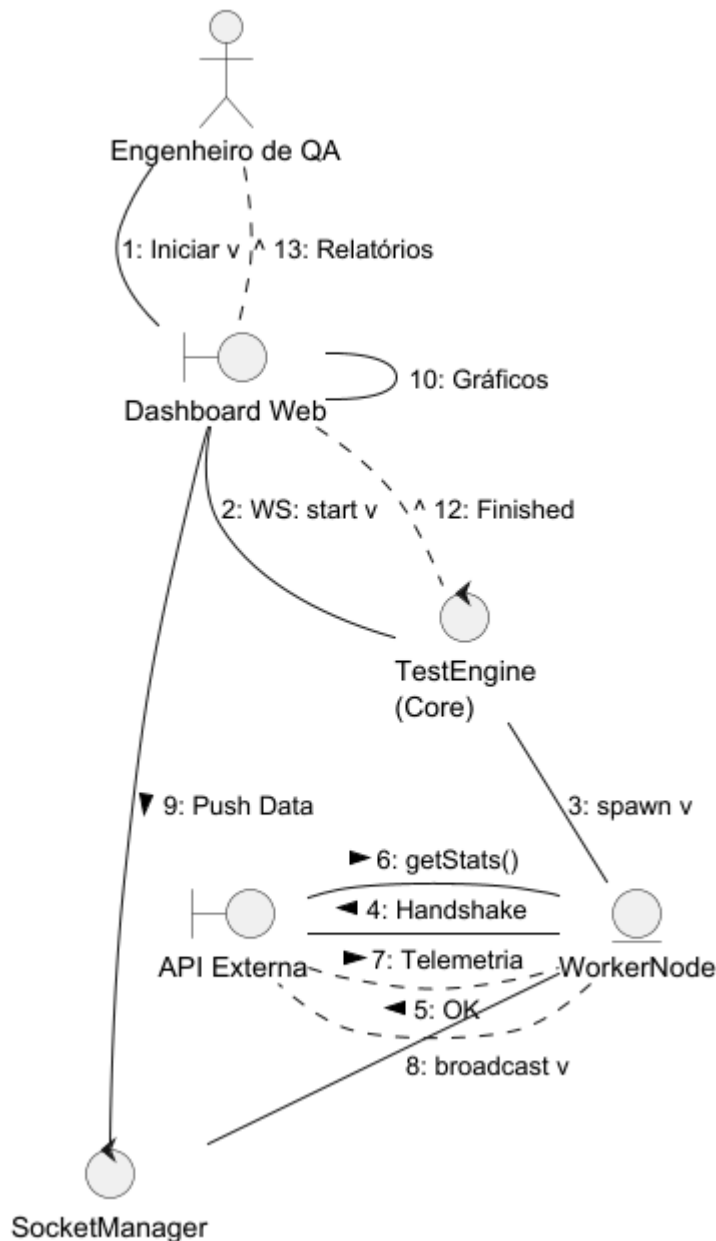


Figura 13: Diagrama de Comunicação de Execução e Telemetria

O diagrama de comunicação de Processamento de Relatórios detalha o "caminho frio" (cold path), focado na consolidação e transformação de dados históricos. O processo é disparado sob demanda pela interface de Relatórios, acionando o Core para executar consultas de agregação complexas no Banco de Dados, em vez de leituras simples. As mensagens evidenciam o desacoplamento arquitetural através do componente ReportGenerator, que aplica uma estratégia de formatação (JSON ou CSV) sobre o dataset retornado antes de gerar o buffer de download. Essa segregação demonstra que a lógica de análise de dados é tratada de forma independente do núcleo de execução, garantindo a integridade e a performance do sistema.

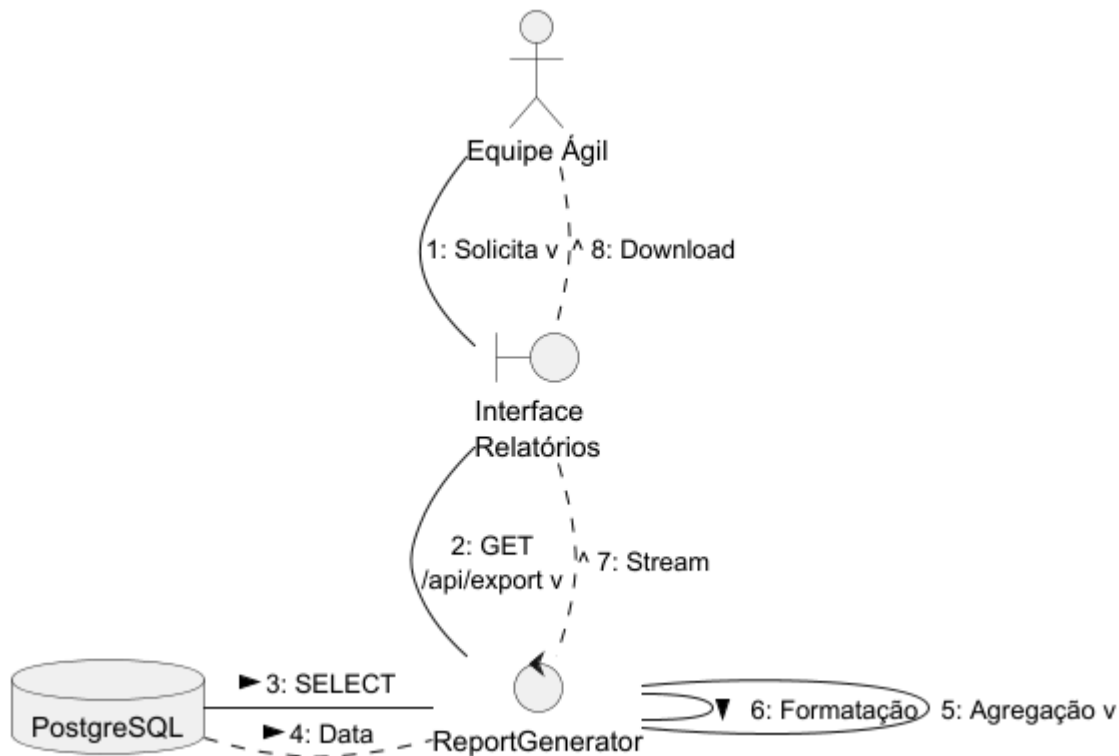
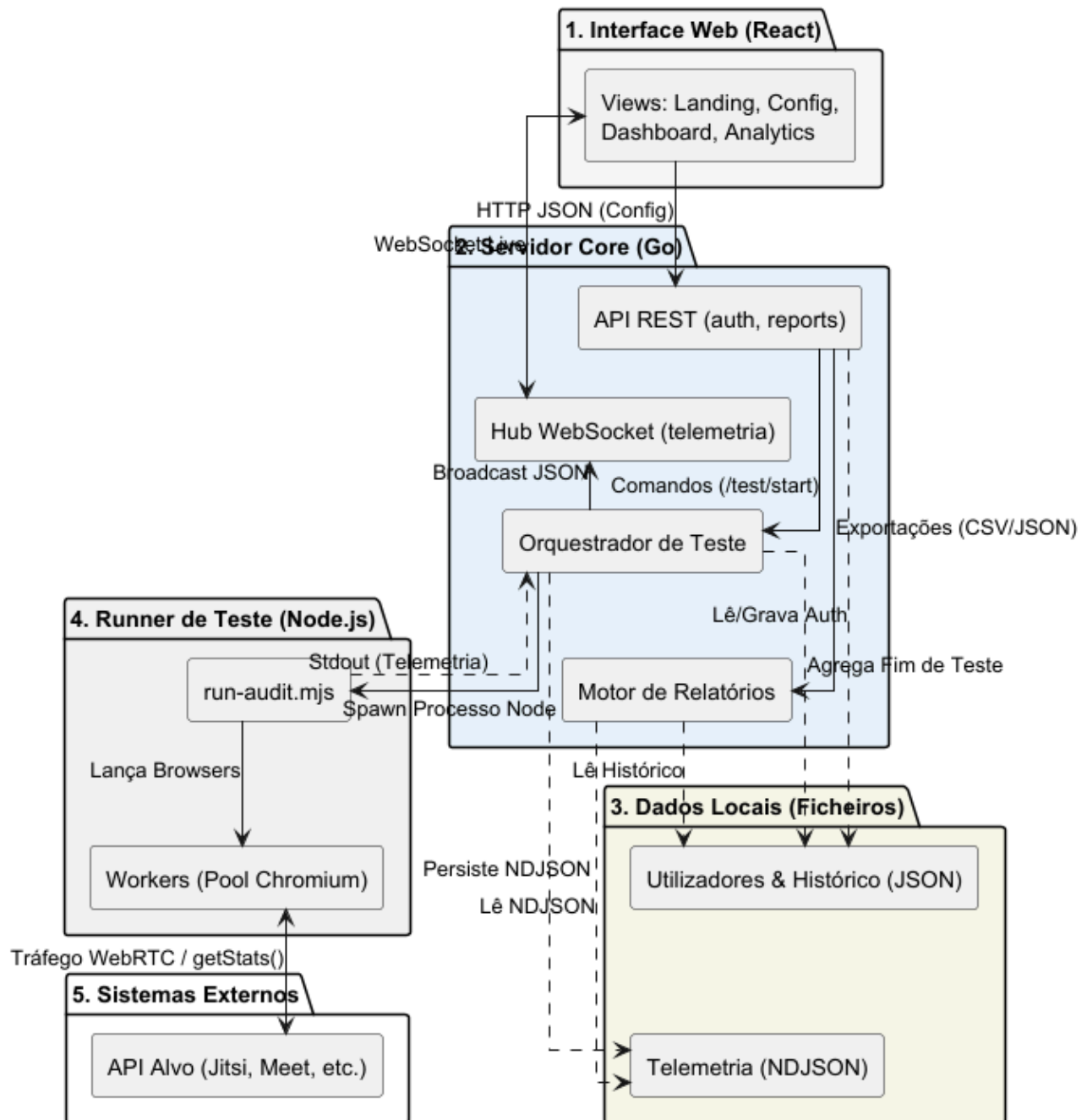


Figura 14: Diagrama de Comunicação de Consolidação de Relatório

3.4 Arquitetura Lógica

A arquitetura lógica do Stream Sentry é representada por um diagrama de componentes que organiza o sistema em camadas de Interface Web (React), Orquestração (Node.js), Execução (Puppeteer) e Dados, priorizando o isolamento de processos e a telemetria em tempo real. A camada de interface gerencia a experiência do usuário através de componentes como o Dashboard Live e o Configurador de Testes, que interagem com o backend via APIs REST e WebSockets. No núcleo de orquestração, o Test Engine coordena o ciclo de vida das simulações, acionando o Worker Cluster para instanciar múltiplos processos Puppeteer que executam as sessões de vídeo e a injeção de instabilidades de rede. A telemetria é processada em um caminho de dados duplo: o "Hot Path" transmite métricas raw instantaneamente do Worker para o Dashboard via WebSocket Hub, enquanto o "Cold Path" persiste as amostras em uma base PostgreSQL para posterior agregação pelo Report Generator. A integração com provedores externos como WebRTC, Zoom e Jitsi ocorre de forma desacoplada através de drivers modulares, garantindo que o sistema atenda aos requisitos de escalabilidade e interoperabilidade definidos no Documento de Visão.

Stream Sentry - Arquitetura Lógica (Formato Retrato A4)**Figura 14: Diagrama de Arquitetura Lógica**

3.5 Diagrama de Atividade

O Diagrama de Atividades (Figura 15) detalha o ciclo de vida operacional do Stream Sentry, desde o controle de acesso até a entrega dos resultados analíticos. O fluxo inicia-se obrigatoriamente pela camada de Segurança (UC00), onde a validação de credenciais via token JWT libera o acesso ao Dashboard. Na fase de Configuração, o sistema valida os parâmetros de estresse e o provedor de vídeo através do Strategy Pattern antes da persistência no banco de dados. Durante a Execução, o diagrama destaca o processamento paralelo (fork): enquanto o motor de testes orquestra os Workers Puppeteer e injeta instabilidades de rede, o sistema gerencia simultaneamente o Hot Path (telemetria em tempo real via WebSockets) e o Cold Path (persistência de amostras para histórico). Ao atingir o tempo de duração estipulado, o processo de teardown é acionado, finalizando as instâncias virtuais e disparando o ReportGenerator para a consolidação das métricas de QoE e geração dos relatórios estruturados.

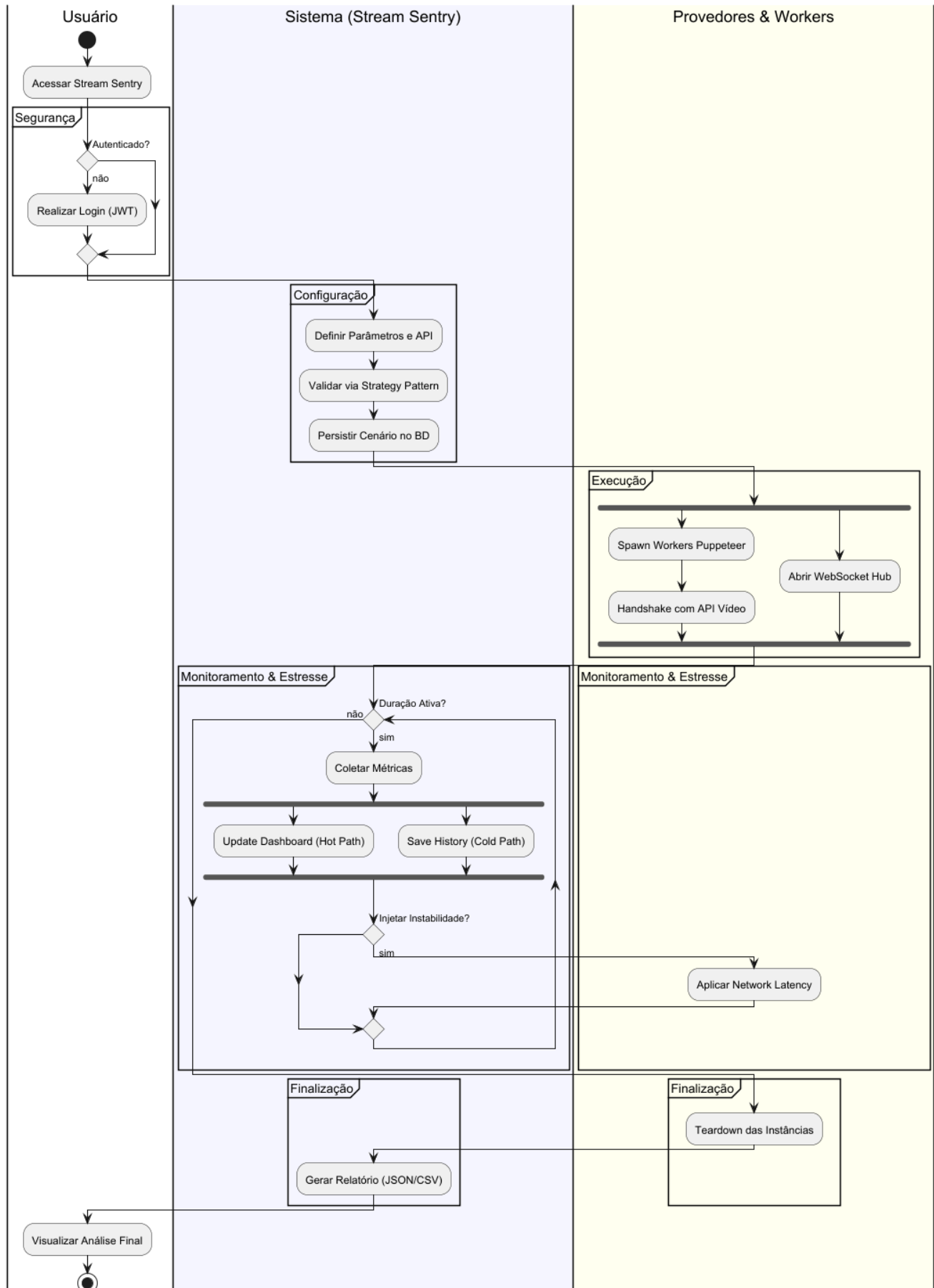


Figura 15: Diagrama de Atividade

3.6 Diagrama de Componentes e Diagrama de Implantação

O Diagrama de Componentes do Stream Sentry (Figura 13) modela os módulos reutilizáveis em três camadas: Interface Web (React) com componentes de UI (Dashboard, Configurar Teste, Executar Teste, Relatórios e Documentação), Stream Sentry Core (Node.js) com controladores REST, serviços (Test Engine, Metric Collector, Report Generator) e persistência via PostgreSQL, e Integração Externa com Jitsi, Zoom SDK e WebRTC. As interfaces REST, WebSocket e SQL garantem o desacoplamento, permitindo comunicação clara e reutilização. Essa estrutura promove manutenibilidade e escalabilidade, alinhada ao Documento de Visão e aos fluxos de dados.

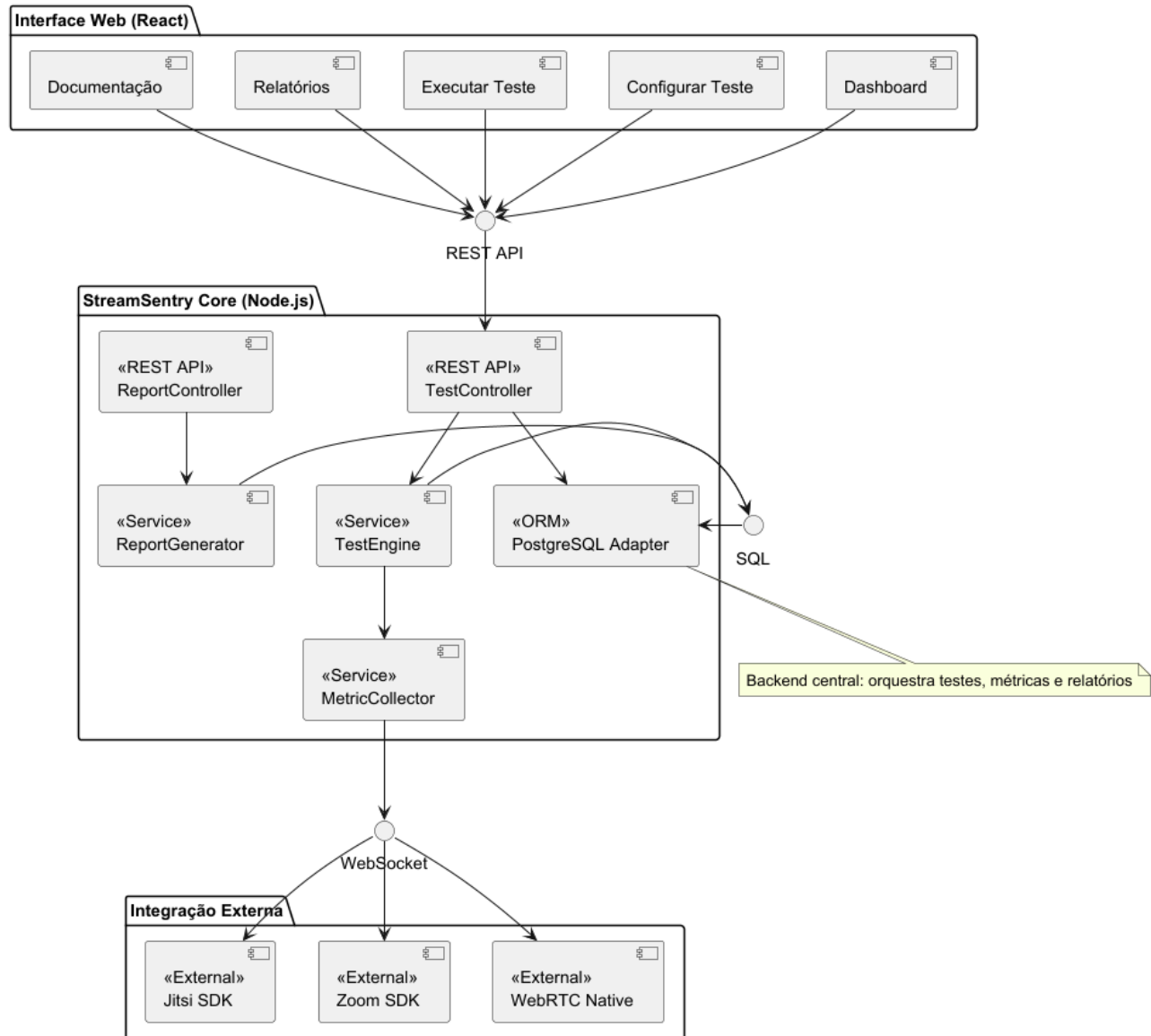
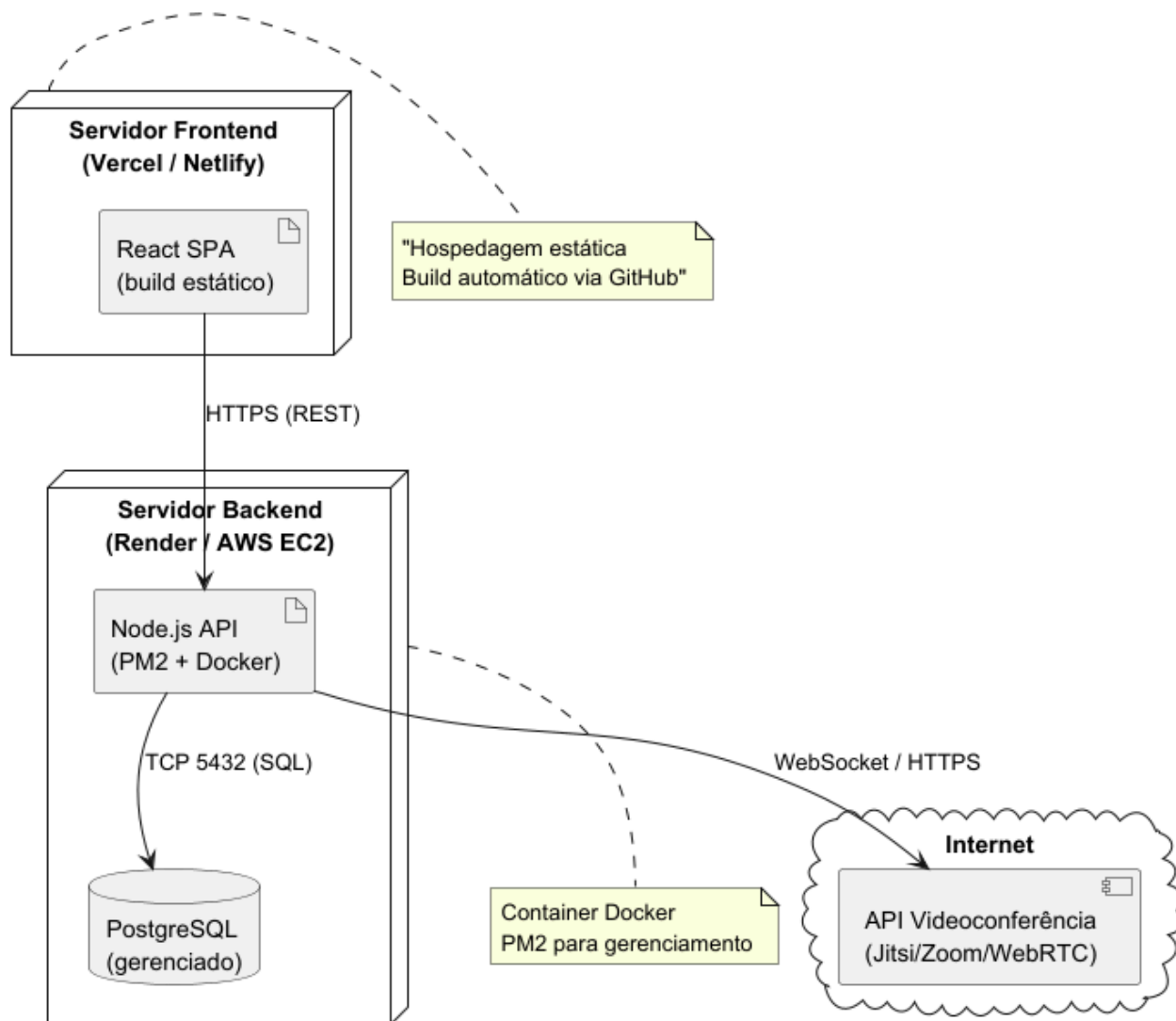


Figura 16: Diagrama de Componentes

O Diagrama de Implantação do Stream Sentry (Figura 14) define a infraestrutura física: o frontend (React SPA) é implantado em Vercel ou Netlify com build automático via GitHub, o backend (Node.js API) roda em Render ou AWS EC2 com container Docker e PM2 para alta disponibilidade, e o banco PostgreSQL é gerenciado (RDS ou Render). A comunicação ocorre via HTTPS (REST) entre frontend e backend, TCP 5432 (SQL) para persistência e WebSocket/HTTPS com APIs externas (Jitsi, Zoom, WebRTC). Essa arquitetura cloud-native garante escalabilidade, segurança e facilidade de deploy, atendendo aos requisitos de automação e integração do sistema.

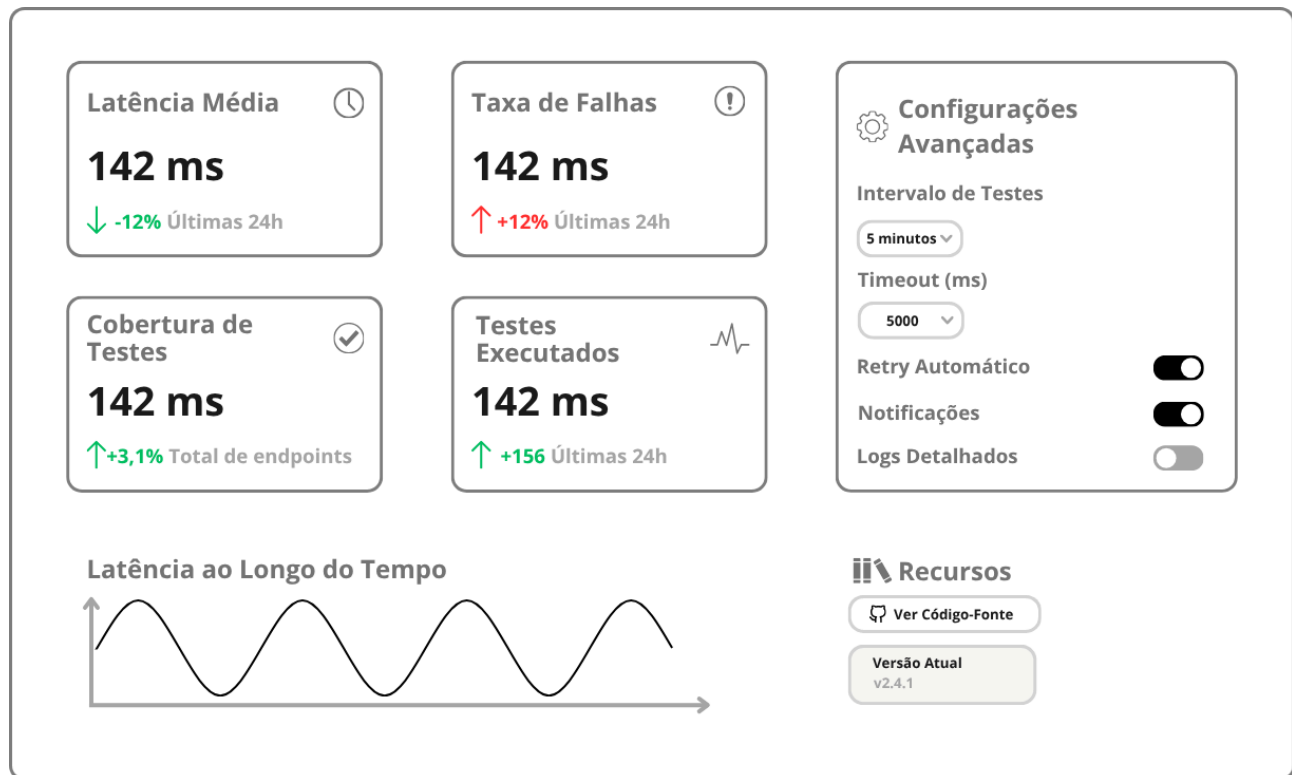
**Figura 17: Diagrama de Implantação**

4. Projeto de Interface com Usuário

O projeto de interface com usuário do Stream Sentry prioriza clareza técnica, usabilidade ágil e consistência visual, oferecendo uma experiência fluida para todos os atores (desenvolvedores, QA, pesquisadores e equipes ágeis). As interfaces comuns incluem um Dashboard central com visão geral de métricas, menu de navegação intuitivo e exportação de relatórios, enquanto telas específicas como Configurar Teste, Executar Teste, Relatórios, Documentação Técnica, Gerenciar Usuários, Histórico, Configurações Avançadas, Suporte e Login/Registro são projetadas com layouts responsivos, validação em tempo real, temas técnicos (modo escuro, terminais) e integração com GitHub. O design é minimalista, funcional e inspirado em ferramentas modernas (React Docs), garantindo acessibilidade, rastreabilidade e foco na automação de testes multimídia, com figuras ilustrativas que reforçam a coerência entre funcionalidade e experiência do usuário.

4.1 Interfaces Comuns a Todos os Atores

A tela inicial do sistema Stream Sentry oferece uma visão geral das operações e métricas de teste, com um menu superior contendo as opções Configurar Teste, Executar Teste, Relatórios e Documentação, um painel central exibindo gráficos e indicadores de desempenho como latência média, taxa de falhas e cobertura de testes, uma barra lateral com atalhos para configurações avançadas e acesso ao código-fonte, além de um botão de exportação para gerar relatórios em JSON ou CSV, tudo em um layout minimalista, responsivo e focado em clareza e usabilidade técnica.

**Figura 18: Tela Principal – Dashboard**

Esta tela permite ao usuário definir os parâmetros de um novo teste automatizado, com campos para selecionar a quantidade de usuários virtuais (via controle deslizante), a API de videoconferência (WebRTC, Jitsi ou Zoom SDK) e a duração do teste (em minutos), exibindo indicadores de status como API conectada e usuários simulados prontos, além de botões “Salvar Configuração” e “Iniciar Teste” para execução direta, tudo em um design simples, com validação rápida de entradas e otimizado para fluxos ágeis.

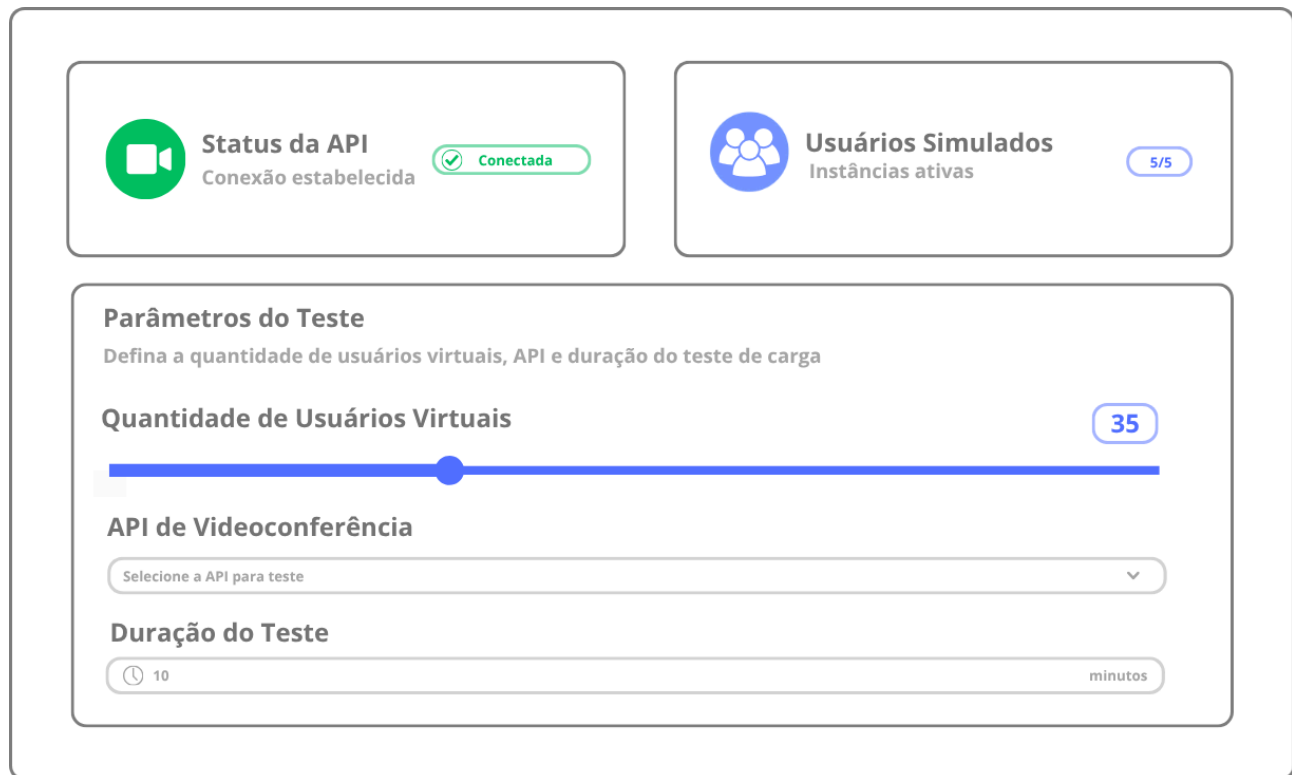


Figura 19: Tela “Configurar Teste”

Esta tela é responsável por monitorar a execução de testes em tempo real, exibindo um painel de logs com informações de desempenho como latência, bitrate e falhas, além de um gráfico dinâmico que mostra variações ao longo do tempo, uma barra lateral de progresso indicando o status da execução e botões para pausar, finalizar ou gerar relatório, tudo em um visual com tema escuro inspirado em terminais técnicos, transmitindo precisão e controle.

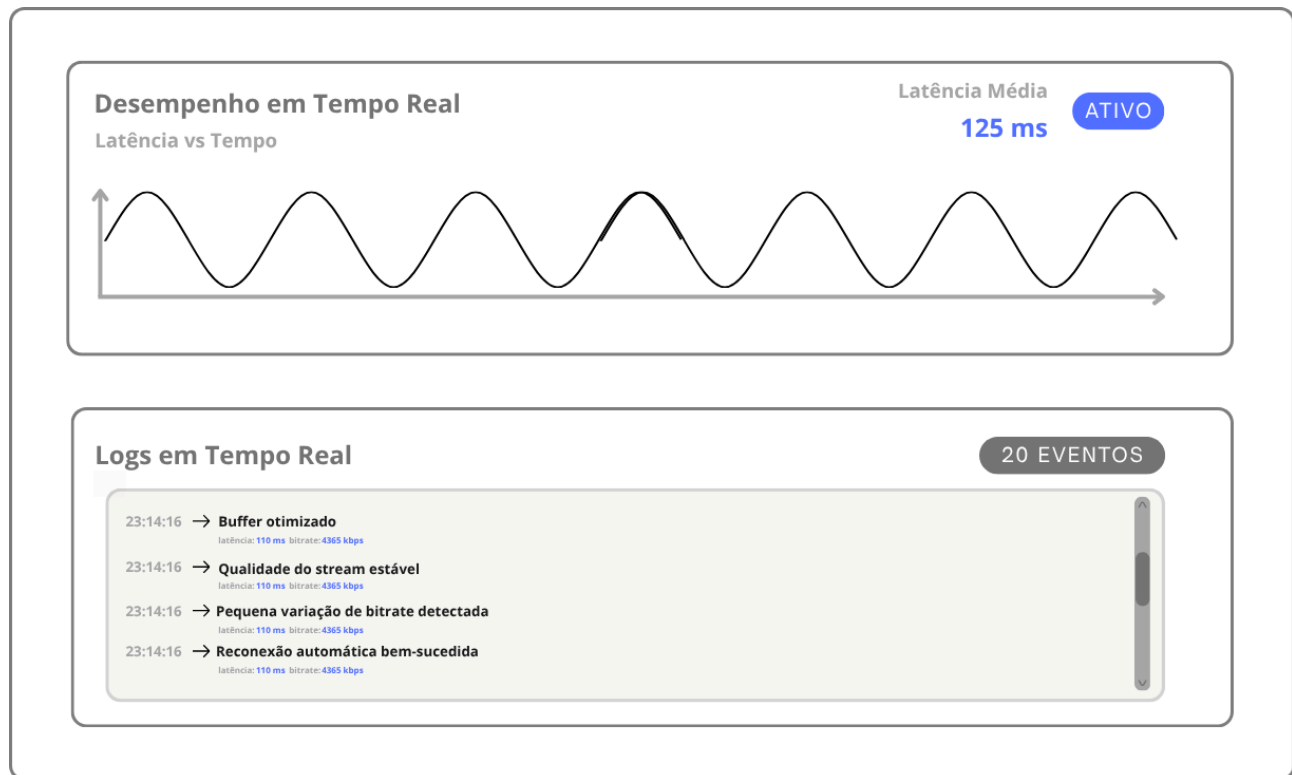


Figura 20: Tela “Executar Teste”

Esta tela centraliza o acesso aos resultados dos testes realizados, apresentando uma tabela com histórico de execuções incluindo API utilizada, número de usuários e status (Sucesso/Falha), exibindo ao selecionar um teste gráficos detalhados com métricas de latência, falhas e qualidade de vídeo/áudio, com opções para filtrar resultados por data, tipo de API ou status e exportar relatórios em JSON ou CSV, tudo em um layout de painel analítico com clareza visual e foco na interpretação rápida de dados.

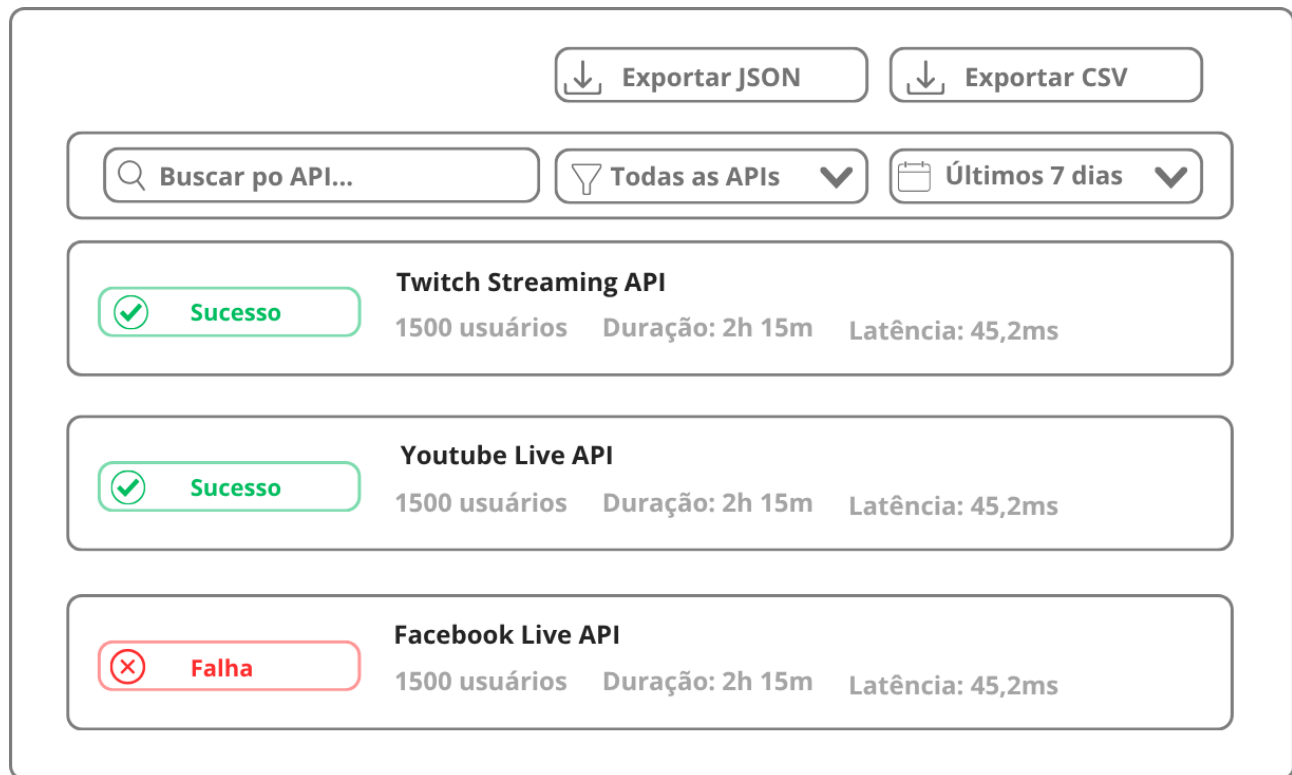


Figura 21: Tela “Relatórios”

Esta tela exibe uma linha do tempo vertical com todos os testes realizados, API usada, duração e status visual (ícones de sucesso/falha), com cada item contendo um link direto para o relatório detalhado, em um estilo inspirado em pipelines de integração contínua como o GitHub Actions, transmitindo automação e rastreabilidade.

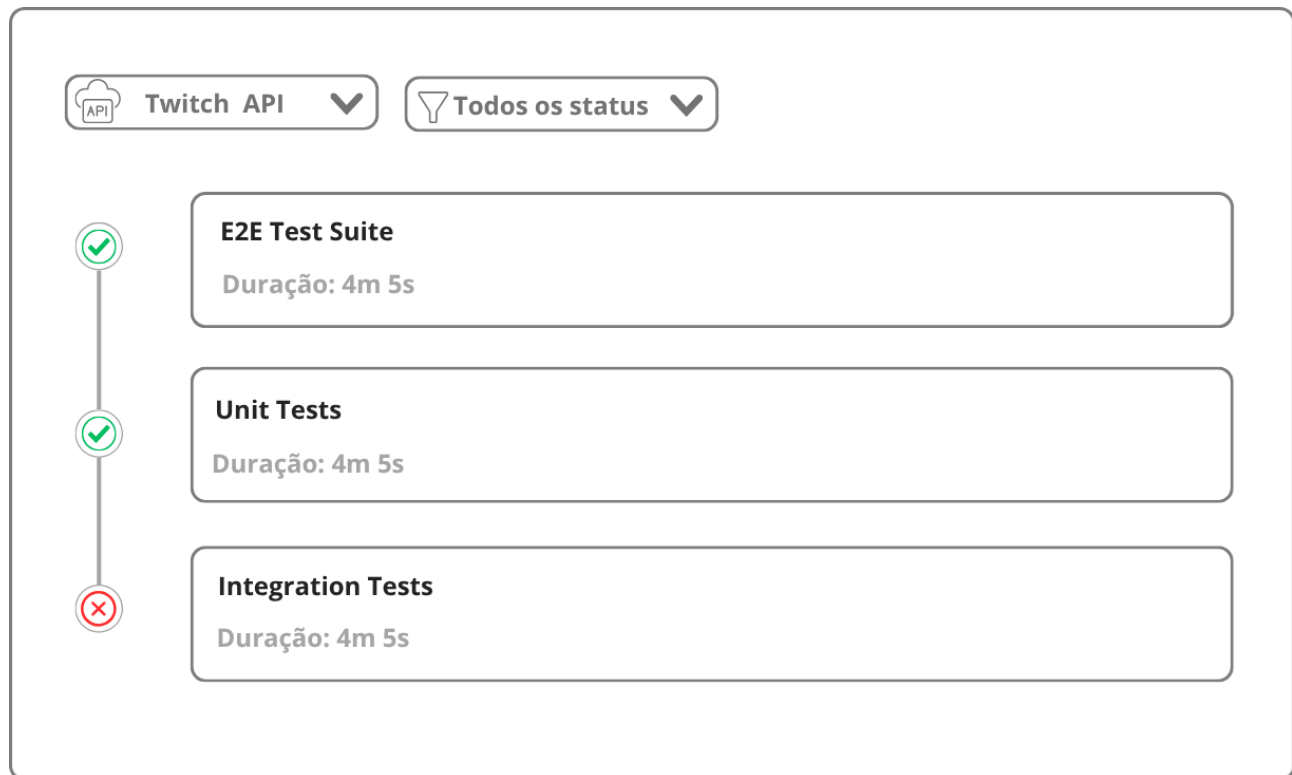


Figura 22: Tela “Histórico de Testes”

Esta tela permite ajustar parâmetros técnicos do sistema, incluindo campos para definir endpoints de APIs, tokens de autenticação e limites de usuários virtuais simultâneos, com um botão “Salvar” para aplicar as alterações, organizada em blocos separados por categoria para priorizar configuração rápida e segura.

**Endpoint da API**
Configure o endpoint da API de videoconferência

URL do Endpoint



**Token de Autenticação**
Configure as credenciais de acesso à API

Token de acesso

**Usuários Virtuais**
Defina o número máximo de usuários virtuais simultâneos

Limite Máximo	Atual	Em Uso	Disponível
100	100	23	77

 **Salvar**

Figura 23: Tela “Configurações Avançadas”

Esta tela de login e registro apresenta um design minimalista com fundo neutro, logo centralizada e campos de autenticação (email e senha), e confirmação de senha, com botões “Entrar” em estilo uniforme e feedback visual claro, visando facilitar o acesso rápido ao sistema e manter coerência visual com o restante da aplicação.

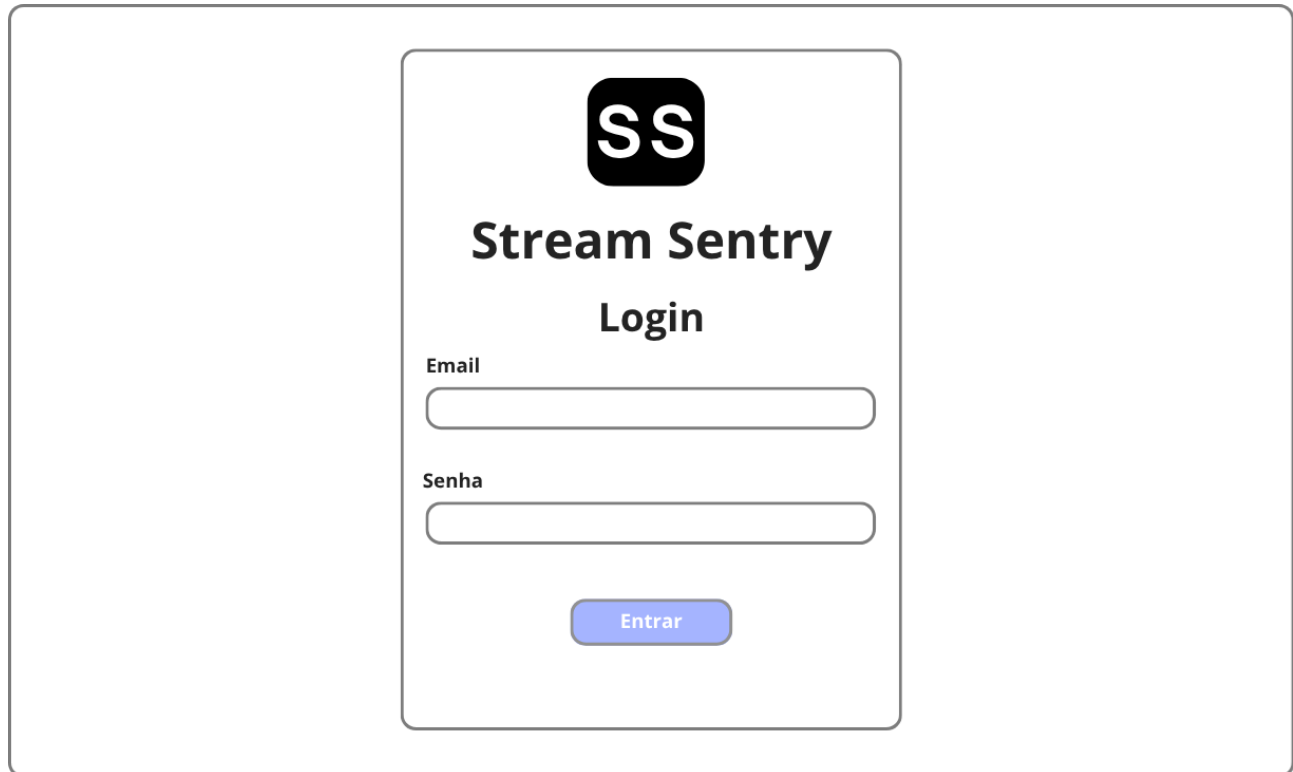
The image shows a mockup of a login screen for a system called 'Stream Sentry'. The screen has a light gray background. In the center, there is a white rounded rectangle containing the login form. At the top of this rectangle is a black square logo with the white letters 'SS'. Below the logo, the text 'Stream Sentry' is written in a bold, black, sans-serif font, followed by 'Login' in a slightly smaller, bold, black, sans-serif font. Below the title, there are two input fields. The first is labeled 'Email' in a small, gray, sans-serif font, and the second is labeled 'Senha' in the same font. Both labels are positioned to the left of their respective input fields. The input fields are white with a thin gray border and rounded corners. Below the input fields, there is a blue button with the word 'Entrar' in white, sans-serif font. The button has rounded corners and a slight gradient.

Figura 24: Tela “Login e Registro”

5. Modelos de Projeto

A seção de Modelos de Projeto apresenta os artefatos detalhados que sustentam a arquitetura e a implementação do Stream Sentry, garantindo consistência entre o Documento de Visão, os diagramas UML e a base de código. O modelo de dados é relacional e formalizado pelo Diagrama de Entidade e Relacionamento (DER) (Figura 25), composto por quatro entidades principais — Usuario, Teste, Relatorio e ConfiguracaoAPI — com relacionamentos 1:N e 1:1, chaves primárias UUID e tipos ENUM para integridade e eficiência. Esse modelo suporta todas as operações do sistema (configurar, executar, relatar) e será implementado em PostgreSQL com ORM (TypeORM/Prisma), sendo consistente com o Diagrama de Classes (Seção 3.1). O glossário de atributos (Tabela 1) lista todos os campos reconhecidos nas entradas (configuração de testes) e saídas (relatórios), com formato, restrições e descrição clara, facilitando a validação de dados e a comunicação entre frontend, backend e banco. Juntos, esses modelos promovem normalização, manutenibilidade e escalabilidade, atendendo aos requisitos funcionais e não funcionais do sistema.

5.1 Modelo de Dados

O modelo de dados do Stream Sentry é relacional e representado pelo Diagrama de Entidade e Relacionamento (DER) (Figura 16), composto por quatro entidades principais: Usuario (com atributos de autenticação e função), Teste (configuração e execução com status gerenciado por flags), Relatório (resultados e arquivos exportados) e Configuração API (credenciais de integração). Os relacionamentos são 1:N entre Usuario e Teste, 1:1 entre Teste e Relatório, e 1:N entre Usuario e Configuração API. O uso de chaves primárias UUID, chaves estrangeiras e tipos ENUM garante integridade referencial, normalização e eficiência em consultas. O modelo suporta todas as operações do sistema (configurar, executar, relatar), é consistente com o Diagrama de Classes (Seção 3.1) e será implementado em PostgreSQL com ORM (TypeORM/Prisma).

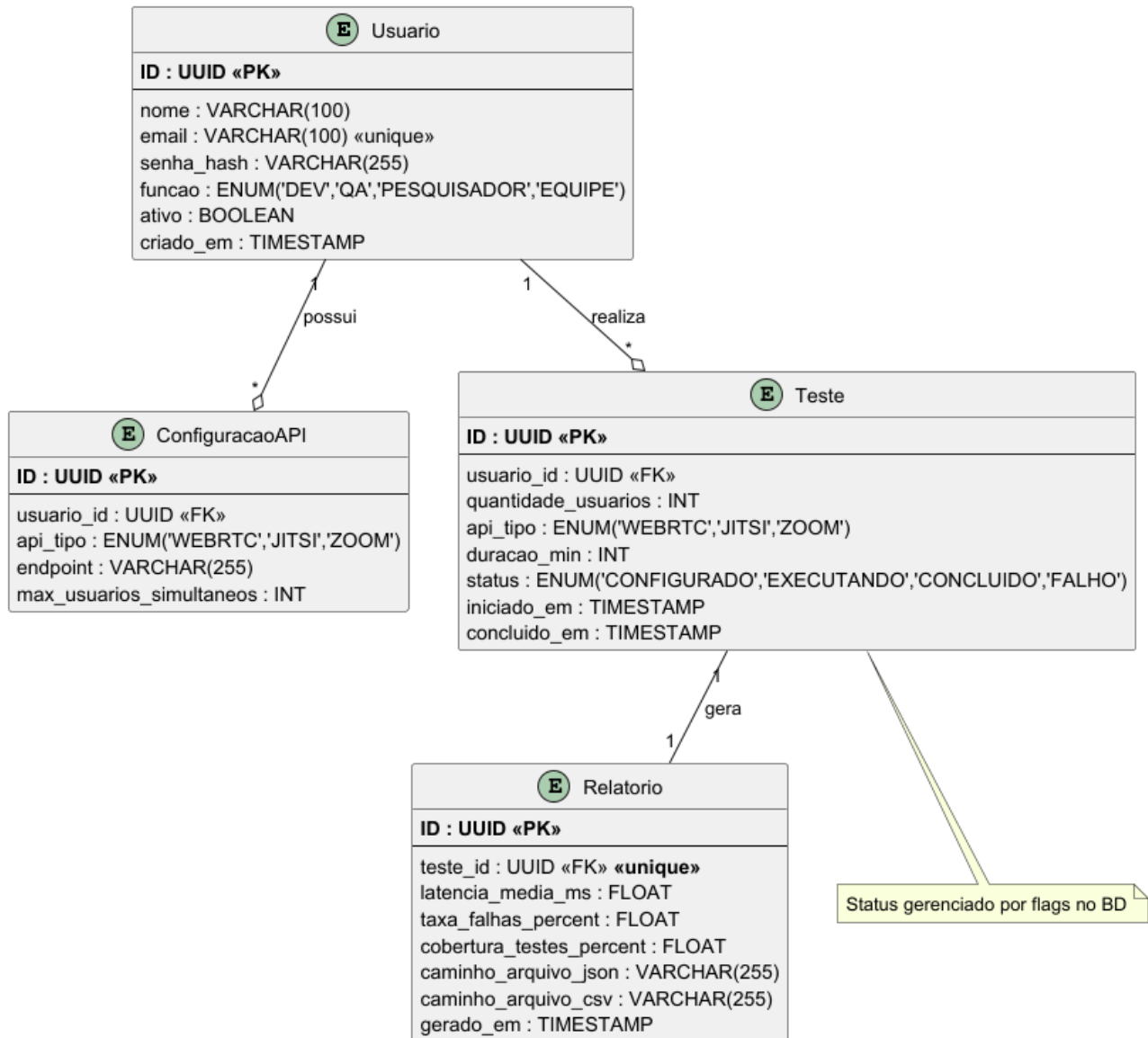


Figura 25: Diagrama de Entidade e Relacionamento

5.2 Glossário

O Glossário de Atributos (Tabela 4) formaliza a definição técnica de todos os campos de dados manipulados pelo sistema Stream Sentry, servindo como uma referência central para o desenvolvimento do schema do banco de dados e para a validação de interfaces. A tabela abrange desde os parâmetros essenciais de configuração dos testes, como `quantidade_usuarios` e `api_tipo`, até as métricas críticas de desempenho coletadas em tempo real, como `latencia_media_ms` e `taxa_falhas_percent`. Para cada atributo, são especificados o formato de dados (incluindo tipos primitivos como INT e FLOAT, bem como ENUMs para controle de estado), as restrições de domínio (como o limite de 1 a 10 usuários virtuais) e a descrição funcional, garantindo a padronização e a integridade referencial das informações que transitam entre o frontend, a API e a camada de persistência.

Atributo	Formato	Descrição
<code>quantidade_usuarios</code>	INT (1–10)	Número de usuários virtuais simulados no teste.
<code>api_tipo</code>	ENUM('WEBRTC','JITSI','ZOOM')	Tipo de API de videoconferência utilizada no teste.
<code>duracao_min</code>	INT (1–1440)	Duração do teste em minutos.
<code>status</code>	ENUM('CONFIGURADO','EXECUTANDO','CONCLUIDO','FALHO')	Estado atual do teste no sistema.
<code>latencia_media_ms</code>	FLOAT	Média de latência de rede durante o teste (em milissegundos).
<code>taxa_falhas_percent</code>	FLOAT (0.0–100.0)	Percentual de falhas de conexão ou pacotes perdidos.
<code>cobertura_testes_percent</code>	FLOAT (0.0–100.0)	Percentual de funcionalidades da API cobertas pelo teste.

Atributo	Formato	Descrição
caminho_arquivo_json	VARCHAR(255)	Caminho ou URL do relatório gerado em formato JSON.
caminho_arquivo_csv	VARCHAR(255)	Caminho ou URL do relatório gerado em formato CSV.
endpoint	VARCHAR(255)	URL base da API de videoconferência.
token	TEXT	Token de autenticação para APIs que exigem (Zoom, Jitsi custom).
max_usuarios_simultaneos	INT	Limite máximo de usuários virtuais por execução.
iniciado_em	TIMESTAMP	Data e hora do início da execução do teste.
concluido_em	TIMESTAMP	Data e hora do término da execução do teste.
gerado_em	TIMESTAMP	Data e hora da geração do relatório.

Tabela 4: Glossário

6. Plano de Testes

Esta seção detalha a estratégia de garantia de qualidade do *Stream Sentry*, dividida em testes de aceitação, focados na validação das necessidades de negócio descritas no Documento de Visão, e testes de integração, focados na comunicação entre os componentes do sistema e serviços externos.

6.1 Plano de Teste de Aceitação

O Plano de Teste de Aceitação tem como objetivo validar se as funcionalidades entregues correspondem às necessidades de negócio definidas no Documento de Visão.

A primeira necessidade abordada refere-se à Configuração Flexível e Automação de Testes, cujo objetivo é garantir a parametrização correta para diferentes provedores de vídeo, respeitando restrições de entrada e segurança. Conforme detalhado na Tabela 5, o cenário TA-01 verifica o fluxo positivo de uma configuração válida no Jitsi para iniciar instâncias virtuais, enquanto o TA-02 aplica testes de fronteira (*Boundary Value*) para impedir que o usuário exceda o limite físico de 500 usuários por instância. Simultaneamente, a segurança é testada no cenário TA-03, que submete credenciais inválidas à API do Zoom para assegurar que o sistema trate adequadamente os erros de autenticação (*Unauthorized*).

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-01	Configuração de Teste Jitsi com Sucesso	Usuário logado; API Jitsi disponível.	API: Jitsi Meet Usuários: 10 Duração: 5 min Ação: Clicar em "Iniciar Teste"	O sistema deve validar os inputs, redirecionar para a tela "Executando", exibir o <i>status</i> "Iniciando Workers" e confirmar a criação de 10 instâncias virtuais.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-02	Validação de Limite de Usuários (Boundary Value)	Usuário na tela de configuração.	API: WebRTC Native Usuários: 501 (Limite é 500) Duração: 2 min	O botão "Iniciar" deve ser desabilitado ou exibir erro imediato: <i>"O limite máximo de usuários por instância é 500"</i> . O teste não deve iniciar.
TA-03	Tratamento de Credenciais Inválidas (Zoom SDK)	Usuário possui <i>token</i> expirado ou inválido.	API: Zoom SDK Token: eyJhbGciOi... (Inválido) Ação: Clicar em "Iniciar Teste"	O sistema deve tentar a conexão, receber erro 401 Unauthorized da API externa e exibir <i>toast</i> de erro: <i>"Falha na autenticação: Verifique seu Token JWT"</i> .

Tabela 5: Configuração e Automação

A segunda necessidade foca no Monitoramento de Qualidade e Coleta de Métricas, visando assegurar a precisão da telemetria em tempo real e o alerta sobre instabilidades. Para validar esses requisitos (Tabela 6), o teste TA-04 simula picos de latência para confirmar a alteração visual dos indicadores de saúde para o estado crítico. Já o cenário TA-05 verifica a resiliência dos contadores de usuários ativos e falhas diante de perdas de pacotes ou desconexões forçadas. O ciclo de vida correto do teste é garantido pelo caso TA-06, que confirma o encerramento automático (*teardown*) das instâncias Puppeteer e a liberação do relatório exatamente ao término da duração estipulada.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-04	Visualização de Latência Crítica	Teste em execução.	Simulação: Injeção de latência de rede (500ms) em 20% dos clientes virtuais.	O gráfico de linha "Latência Média" deve apresentar um pico acima de 400ms. O indicador de saúde do sistema deve alterar a cor de Verde para Vermelho (Crítico).
TA-05	Contagem de Falhas de Conexão (Packet Loss)	Teste em execução com 10 usuários.	Simulação: Interrupção forçada da rede em 1 cliente virtual (simulação de queda).	O contador "Usuários Ativos" deve cair para 9 e o contador "Erros de Conexão" deve incrementar em +1. O teste não deve ser abortado para os demais usuários.

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-06	Encerramento Automático e Consistência	Teste configurado para 1 minuto.	Ação: Aguardar o cronômetro atingir 00:00.	O sistema deve forçar o <i>teardown</i> (desconexão) de todos os clientes Puppeteer, alterar o <i>status</i> do teste para "Concluído" e habilitar o botão "Ver Relatório".

Tabela 6: Monitoramento de Qualidade

A terceira necessidade cobre a Análise de Dados e Relatórios, validando a persistência e integridade dos dados históricos para ferramentas externas. Conforme a Tabela 7, o cenário TA-07 assegura que a exportação JSON contenha todas as chaves obrigatórias (*test_id*, *metrics*, *summary*) e dados consistentes. A persistência e ordenação cronológica correta na interface de histórico são verificadas pelo teste TA-08. Adicionalmente, o caso TA-09 valida a funcionalidade de filtragem, garantindo que o usuário consiga isolar visualmente os relatórios na tabela com base no provedor de API utilizado (ex: exibir apenas testes Zoom).

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-07	Integridade da Exportação JSON	Teste concluído com ID #105.	Tela: Detalhes do Relatório Ação: Clicar em "Exportar JSON"	<i>Download</i> do arquivo report_105.json. O arquivo deve ser um JSON válido contendo as chaves: test_id, metrics_array, config, e summary (média de latência e total de falhas).
TA-08	Persistência no Histórico	Teste TA-06 recém-concluído.	Ação: Navegar para a rota /history (Histórico).	O teste mais recente deve aparecer no topo da lista, exibindo corretamente: Data atual, Duração (1 min), API utilizada e Status "Sucesso".

ID	Caso de Teste	Pré-condição	Entrada de Dados	Saída Esperada / Critério de Aceite
TA-09	Filtragem de Relatórios por API	Existem registros de testes Jitsi e Zoom no banco.	Filtro: Selecionar "Zoom SDK" no <i>dropdown</i> de filtros.	A tabela deve recarregar exibindo apenas as linhas onde a coluna "Provedor" é igual a "Zoom". Registros de Jitsi devem ser ocultados.

Tabela 7: Análise e Relatórios

6.2 Plano de Teste de Integração

O teste de integração foca na comunicação entre os módulos internos do Stream Sentry e os serviços externos, assegurando que as unidades de software interajam conforme o esperado. Dada a arquitetura do sistema baseada em microsserviços lógicos, a estratégia adotada será a Incremental Bottom-Up, iniciando a validação pela camada de persistência e lógica de negócios (backend e banco de dados) antes de avançar para a integração com a interface de usuário. Essa abordagem permite isolar falhas de comunicação em níveis mais baixos antes de testar fluxos completos na camada de apresentação.

Para operacionalizar essa estratégia, a execução utiliza contêineres Docker para isolar o Banco de Dados, garantindo um ambiente efêmero e limpo para cada bateria de testes. No que tange às APIs externas (Jitsi e Zoom), serão utilizados *Mocks* e bibliotecas como Nock dentro do pipeline de Integração Contínua (CI) para simular respostas HTTP e WebSocket, evitando custos operacionais e a instabilidade inerente à rede externa. As interfaces críticas e os cenários validados neste processo, incluindo a comunicação REST, transações SQL e canais WebRTC, estão detalhados na Tabela 8 a seguir.

ID	Interface Envolvida	Descrição do Cenário	Validação Técnica
TI-01	I-02 (Backend ↔ DB)	Persistência de Configuração O <i>Controller</i> recebe um POST e deve salvar no PostgreSQL.	Enviar POST para /api/tests. Verificar via <i>query</i> SQL direta (SELECT * FROM tests WHERE id = ...) se o registro foi criado com os dados corretos.
TI-02	I-03 (Engine ↔ Jitsi)	Handshake de Sala de Vídeo O <i>Worker</i> deve conseguir entrar em uma sala Jitsi real ou mockada.	O script do Puppeteer deve identificar o elemento DOM #jitsiConferenceFrame (indicando sucesso no carregamento do iframe da API externa) e retornar <i>status code</i> 200.

ID	Interface Envolvida	Descrição do Cenário	Validação Técnica
TI-03	I-01 (Front ↔ Back)	Atualização de Métricas via Socket O Backend emite eventos de telemetria que o Frontend deve consumir.	Emitir evento <code>metric_update</code> no <i>socket</i> do servidor. Verificar se o estado do componente React (<code>useState</code>) é atualizado e se o gráfico renderiza o novo ponto de dados.
TI-04	I-01 e I-02	Fluxo Completo de Relatório Frontend solicita relatório histórico ao Backend.	Requisição GET <code>/api/reports/:id</code> . O Backend deve realizar o <i>JOIN</i> nas tabelas Test e Metrics, serializar o JSON e o Frontend deve renderizar os dados sem erros de tipagem (<code>undefined</code>).

Tabela 8: Integração e Estratégia

7. Cronograma e Processo de Implementação

Esta seção descreve a abordagem de engenharia de software adotada para a construção do *Stream Sentry*, detalhando a metodologia, as ferramentas de controle, o pipeline de integração contínua (CI/CD) e o planejamento temporal para garantir a entrega do projeto dentro do prazo estipulado.

7.1 Cronograma

O cronograma do TCC II será executado em um modelo Ágil, estruturado em dez Sprints de duas semanas cada, cobrindo o desenvolvimento completo do software e a documentação técnica até o final de junho de 2026. Este planejamento prioriza o objetivo central do TCC: as Sprints iniciais (S1 a S5) focam no Core, Métricas e Integrações com todas as APIs de vídeo. As Sprints seguintes (S6 a S8) dedicam-se à Interface Web e aos Testes Integrados (E2E). Por fim, as Sprints finais (S9 e S10)

são reservadas à Coleta de Métricas Definitivas e à Redação dos Capítulos de Resultados da monografia, garantindo que o produto técnico e os dados para o TCC estejam consolidados no prazo.

Mês	Período (Quinzenas)	Sprint	Foco Principal	Atividades Detalhadas
Fev/26	1ª Quinzenal (1-15)	S1	Setup e Core Base	Configuração do ambiente (Node.js/PostgreSQL). Implementação dos modelos de dados e do TestController.
	2ª Quinzenal (16-29)	S2	Motor de Testes	Implementação do TestEngine e MetricCollector base. Primeira execução funcional do teste com WebRTC Native.
Mar/26	1ª Quinzenal (1-15)	S3	Coleta de Métricas	Finalização do MetricCollector. Implementação das rotinas de coleta de latência e taxa de falhas e persistência no BD.
	2ª Quinzenal (16-31)	S4	Integração de APIs	Implementação da conexão com Zoom SDK e Jitsi SDK . Validação da coleta de métricas nas APIs integradas.
Abr/26	1ª Quinzenal (1-15)	S5	Relatórios RAW e Docs	Implementação do ReportGenerator. Geração do Relatório em formato RAW (JSON/CSV). Finalização do Capítulo 3 (Metodologia).

	2ª Quinzenal (16-30)	S6	Interface Essencial	Desenvolvimento das telas Configurar Teste e Executar Teste (monitoramento em tempo real) no React.
Mai/26	1ª Quinzenal (1-15)	S7	Visualização de Dados	Desenvolvimento das telas Relatórios e Dashboard no React, incluindo visualização de gráficos e exportação de dados.
	2ª Quinzenal (16-31)	S8	Testes Integrados (E2E)	Execução da primeira rodada de Testes Integrados (E2E) completos em todos os fluxos e APIs. Correção de <i>bugs</i> críticos.
Jun/26	1ª Quinzenal (1-15)	S9	Qualidade e Docs Técnica	Segunda rodada de Testes Integrados . Elaboração da Documentação Técnica (Arquitetura, APIs) e do Capítulo 4 (Resultados/Análise V1).
	2ª Quinzenal (16-30)	S10	Fechamento e Resultados Finais	Fechamento da Versão 1.2. Coleta de Métricas Definitivas para o TCC. Finalização do Capítulo 5 (Conclusão e Trabalhos Futuros).

Tabela 9: Cronograma

7.2 Metodologia de Desenvolvimento

O desenvolvimento do *Stream Sentry* adota a metodologia Ágil, utilizando uma adaptação do *framework* Scrum para gerenciar o ciclo de vida do software. O trabalho é organizado em *Sprints* quinzenais, o que permite uma entrega incremental de valor e ajustes rápidos de escopo conforme necessário. A estratégia de construção prioriza inicialmente o desenvolvimento do núcleo de execução de testes (MVP), garantindo a estabilidade do *backend*, para posteriormente expandir o foco para a interface de usuário e os módulos avançados de relatórios.

Para a gestão e rastreamento das tarefas, utiliza-se a ferramenta *GitHub Projects* configurada em formato Kanban. O fluxo de trabalho visualiza o progresso das atividades através de colunas de estado, movendo-se de "A Fazer" (*To Do*) para "Em Progresso", "Em Revisão" e finalmente "Concluído" (*Done*). O controle de versão do código-fonte é realizado através do Git, seguindo rigorosamente o modelo *Feature Branch Workflow*. Neste modelo, cada nova funcionalidade ou correção de *bug* é desenvolvida em um ramo (*branch*) isolado. A integração dessas alterações ao ramo principal (*main*) ocorre exclusivamente por meio de *Pull Requests* (PR), que exigem a revisão obrigatória do código por pares antes da aprovação, garantindo a qualidade e a segurança da base de código.

7.3 Ferramentas e Pipeline de CI/CD

Para assegurar a confiabilidade do *software* e automatizar o processo de entrega, foi estabelecido um pipeline de Integração Contínua e Entrega Contínua (CI/CD) utilizando a plataforma *GitHub Actions*. Este mecanismo é fundamental para impedir que códigos com erros de sintaxe ou falhas lógicas sejam integrados ao ambiente de produção, executando uma bateria de verificações automáticas a cada nova submissão de código ao repositório.

O fluxo do pipeline é executado sequencialmente em quatro estágios principais. Inicialmente, ocorre a análise estática e padronização do código (*Linting*) através das ferramentas ESLint e Prettier, que identificam precocemente erros de sintaxe e desvios de estilo. Em seguida, são executados os testes automatizados, que compreendem testes unitários com Jest para validar a lógica dos serviços isolados e scripts de integração para verificar a conectividade com o banco de dados PostgreSQL. Caso os testes sejam bem-sucedidos, o pipeline avança para a etapa de *Build*, onde o *frontend* (React/Vite) e o *backend* (Node/TypeScript) são compilados para garantir a integridade da construção. Por fim, após a aprovação do *Pull Request* na ramificação principal, o processo de *Deploy* (CD) é disparado automaticamente: a interface web é implantada na plataforma Vercel, enquanto o servidor e seus *workers* são containerizados e enviados para o serviço de hospedagem Render ou registro de containers Docker.